

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CÂMPUS CORNÉLIO PROCÓPIO
DIRETORIA DE PESQUISA E PÓS-GRADUAÇÃO
PROGRAMA DE PÓS-GRADUAÇÃO EM INFORMÁTICA

ANDREY VINICIUS ROCHA DE OLIVEIRA

**ALGORITMOS EVOLUTIVOS APLICADOS AO PROBLEMA DA
ÁRVORE DE STEINER EUCLIDIANO**

DISSERTAÇÃO – MESTRADO

CORNÉLIO PROCÓPIO

2019

ANDREY VINICIUS ROCHA DE OLIVEIRA

**ALGORITMOS EVOLUTIVOS APLICADOS AO PROBLEMA DA
ÁRVORE DE STEINER EUCLIDIANO**

Dissertação apresentada ao Programa de Pós-Graduação em Informática da Universidade Tecnológica Federal do Paraná – UTFPR como requisito parcial para a obtenção do título de “Mestre Profissional em Informática”.

Orientador: Danilo Sipoli Sanches

CORNÉLIO PROCÓPIO

2019

Dados Internacionais de Catalogação na Publicação

O48 Oliveira, Andrey Vinicius Rocha de

Algoritmos evolutivos aplicados ao problema da árvore de Steiner Euclidiano / Andrey Vinicius Rocha de Oliveira. - 2019.
70 f. : il. color. ; 31 cm.

Orientador: Danilo Sipoli Sanches.
Dissertação (Mestrado) – Universidade Tecnológica Federal do Paraná. Programa de Pós-Graduação em Informática, Cornélio Procópio, 2019.
Bibliografia: p. 66-70.

1. Algoritmos computacionais. 2. Árvores (Teoria dos grafos). 3. Algoritmos genéticos. 4. Informática – Dissertações. I. Sanches, Danilo Sipoli, orient. II. Universidade Tecnológica Federal do Paraná. Programa de Pós-Graduação em Informática. III. Título.

CDD (22. ed.) 004

Biblioteca da UTFPR - Câmpus Cornélio Procópio

Bibliotecário/Documentalista responsável:
Romeu Righetti de Araujo – CRB-9/1676



Título da Dissertação Nº 62:

“ALGORITMOS EVOLUTIVOS APLICADOS AO PROBLEMA DA ÁRVORE DE STEINER EUCLIDIANO”.

por

Andrey Vinicius Rocha de Oliveira

Orientador: **Prof. Dr. Danilo Sipoli Sanches**

Esta dissertação foi apresentada como requisito parcial à obtenção do grau de MESTRE EM INFORMÁTICA – Área de Concentração: Computação Aplicada, pelo Programa de Pós-Graduação em Informática – PPGI – da Universidade Tecnológica Federal do Paraná – UTFPR – Câmpus Cornélio Procópio, às 14h do dia 05 de agosto de 2019. O trabalho foi _____ pela Banca Examinadora, composta pelos professores:

Prof. Dr. Danilo Sipoli Sanches
(Presidente – UTFPR-CP)

Prof. Dr. Rodrigo Henrique Cunha Palácios
(UTFPR-CP)

Profa. Dra. Telma Woerle de Lima Soares
(UFG)
Participação à distância via _____

Visto da coordenação:

Danilo Sipoli Sanches
Coordenador do Programa de Pós-Graduação em Informática
UTFPR Câmpus Cornélio Procópio

A Folha de Aprovação assinada encontra-se na Coordenação do Programa.

Dedico este trabalho ao meu pai, Gelson A. de Oliveira (in memoriam),
por seu caráter e dignidade.

AGRADECIMENTOS

A minha mãe Neiva R. Oliveira, meu irmão Edson de Oliveira e minha namorada Danieli L. Campos pelo apoio nessa etapa da minha vida.

Ao meu orientador, prof. Dr. Danilo S. Sanches, pela oportunidade oferecida, pela paciência e pelos conhecimentos transmitidos.

Aos professores do programa de pós-graduação em informática (PPGI) pelos conhecimentos transmitidos.

A todos contribuíram direta ou indiretamente para a conclusão deste trabalho.

RESUMO

OLIVEIRA, Andrey. ALGORITMOS EVOLUTIVOS APLICADOS AO PROBLEMA DA ÁRVORE DE STEINER EUCLIDIANO. 72 f. Dissertação – Mestrado – Programa de Pós-graduação em Informática, Universidade Tecnológica Federal do Paraná. Cornélio Procópio, 2019.

Este trabalho apresenta a aplicação de algoritmos evolutivos híbridos para o problema da árvore de Steiner euclidiano. Esse é um problema de otimização cujo objetivo é obter uma árvore de comprimento mínimo para interconectar um conjunto de pontos fixos. Para que esse objetivo seja alcançado é permitido adicionar novos pontos, denominados pontos Steiner. Com a finalidade de obter soluções de boa qualidade em tempo computacional viável, duas heurísticas são utilizadas, uma utiliza o algoritmo evolução diferencial em que são testadas estratégias de mutação e a outra faz uso de um algoritmo genético, em que se investiga os efeitos de operadores de cruzamento e mutação relativamente gulosos, sobreposição de gerações e estratégias evolutivas. Os experimentos são realizados com os problemas da biblioteca *OR-Library* e apresentam resultados próximos aos melhores métodos presentes na literatura.

Palavras-chave: Algoritmos evolutivos, Árvore mínima de Steiner, Evolução diferencial, Algoritmo genético

ABSTRACT

OLIVEIRA, Andrey. EVOLUTIONARY ALGORITHMS APPLIED TO THE EUCLIDEAN STEINER TREE PROBLEM. 72 f. Dissertação – Mestrado – Programa de Pós-graduação em Informática, Universidade Tecnológica Federal do Paraná. Cornélio Procópio, 2019.

This work presents the application of hybrid evolutionary algorithms to the Euclidean Steiner tree problem. This is an optimization problem whose objective is to obtain a minimum length tree to interconnect a set of fixed points. To achieve this goal it is allowed to add new points, called Steiner points. In order to obtain good quality solutions in viable computational time, two heuristics are used, one uses the differential evolution algorithm in which mutation strategies are tested and the other makes use of a genetic algorithm, which investigates the effects of relatively greedy crossover and mutation operators, overlapping of generations, and evolutionary strategies. The experiments are performed with the problems of the OR-Library and present results close to the best methods present in the literature.

Keywords: Evolutionary algorithms, Steiner minimal tree, Differential evolution, Genetic algorithm

LISTA DE FIGURAS

FIGURA 1	– Ponto de Torriceli e linhas de Simpson	15
FIGURA 2	– Figuras de Gergonne	16
FIGURA 3	– Grafo simples	18
FIGURA 4	– Subgrafo	18
FIGURA 5	– Grafo ponderado	19
FIGURA 6	– Árvore	20
FIGURA 7	– Grafo euclidiano	21
FIGURA 8	– Árvore geradora mínima	23
FIGURA 9	– Árvore mínima de Steiner	26
FIGURA 10	– Topologias equivalentes	27
FIGURA 11	– Topologia degenerada	27
FIGURA 12	– Condição de ângulo	28
FIGURA 13	– Decomposição de uma árvore	30
FIGURA 14	– MST e SMT de um triângulo equilátero	31
FIGURA 15	– Processo de steinerizações locais	35
FIGURA 16	– RMST gerada por steinerizações locais em uma MST	36
FIGURA 17	– Diagrama de Voronoi e triangulação de Delaunay	37
FIGURA 18	– Construção da árvore de Steiner no DE	48
FIGURA 19	– Operador de cruzamento para árvores	51
FIGURA 20	– Primeiro operador de mutação (<i>Rem</i>)	53
FIGURA 21	– Segundo operador de mutação (<i>Adj</i>)	53
FIGURA 22	– Estratégias de mutação na instância 40.15	55
FIGURA 23	– Estratégias de mutação na instância 50.01	56
FIGURA 24	– Operadores no GA de estado estacionário. Instância: 100.1	57
FIGURA 25	– Operadores no GA de estado estacionário	58
FIGURA 26	– Operadores no <i>generation gap algorithm</i> . Instância: 100.1	59
FIGURA 27	– Operadores no <i>generation gap algorithm</i>	60
FIGURA 28	– Operadores no <i>generation gap algorithm</i> com redefinição	61
FIGURA 29	– Análise de desempenho	63

LISTA DE SIGLAS

BSTP	Bottleneck Steiner Tree Problem
DE	Differential Evolution
EA	Evolutionary Algorithm
ES	Evolution Strategies
ESTP	Euclidean Steiner Tree Problem
FST	Full Steiner Tree
GA	Genetic Algorithm
MST	Minimum Spanning Tree
NSTP	Network Steiner Tree Problem
RMST	Relatively Minimal Steiner Tree
RSTP	Rectilinear Steiner Tree Problem
SMT	Steiner Minimal Tree
SMTP	Steiner Minimal Tree Problem
SR	Steiner Ratio
ST	Steiner Tree
STP	Steiner Tree Problem
STPG	Steiner Tree Problem in Graphs
VLSI	Very-Large-Scale Integration

SUMÁRIO

1	INTRODUÇÃO	12
1.1	MOTIVAÇÃO	12
1.2	OBJETIVO	13
1.3	ORGANIZAÇÃO DA DISSERTAÇÃO	13
2	A ORIGEM DO PROBLEMA DE STEINER	14
2.1	O PROBLEMA FERMAT-TORRICELLI	14
2.2	O PROBLEMA DE FERMAT GENERALIZADO	15
3	TEORIA DOS GRAFOS	17
3.1	GRAFOS	17
3.2	ÁRVORES	19
3.3	GRAFOS EUCLIDIANOS	20
4	ÁRVORES GERADORAS	22
4.1	ÁRVORE GERADORA MÍNIMA	22
4.2	ALGORITMO DE KRUSKAL	23
4.3	ALGORITMO DE PRIM	24
5	ÁRVORES DE STEINER	25
5.1	DEFINIÇÕES	25
5.2	TIPOS DE ÁRVORES	26
5.3	PROPRIEDADES	28
5.4	RAZÃO DE STEINER	30
5.5	VARIANTES DO PROBLEMA DE STEINER	31
6	MÉTODOS DE RESOLUÇÃO	33
6.1	ALGORITMOS EXATOS	33
6.2	ALGORITMOS EMBASADOS EM ÁRVORES GERADORAS	35
6.3	ALGORITMOS EMBASADOS NA TRIANGULAÇÃO DE DELAUNAY	37
6.4	META-HEURÍSTICAS	38
7	COMPUTAÇÃO EVOLUTIVA	40
7.1	ALGORITMOS EVOLUTIVOS	40
7.2	EVOLUÇÃO DIFERENCIAL	42
7.3	ALGORITMO GENÉTICO	43
8	ALGORITMOS DESENVOLVIDOS	46
8.1	EVOLUÇÃO DIFERENCIAL HÍBRIDO	47
8.2	ALGORITMO GENÉTICO HÍBRIDO	48
9	EXPERIMENTOS COMPUTACIONAIS	54
9.1	EVOLUÇÃO DIFERENCIAL	54
9.1.1	ESTRATÉGIAS DE MUTAÇÃO	54
9.2	ALGORITMO GENÉTICO	57
9.2.1	OPERADORES DE CRUZAMENTO E MUTAÇÃO	57
9.2.2	SOBREPOSIÇÃO DE GERAÇÕES	59
9.2.3	REDEFINIÇÃO POPULACIONAL	60
9.3	ANÁLISE DE DESEMPENHO	61

10 CONSIDERAÇÕES FINAIS	65
REFERÊNCIAS	67

1 INTRODUÇÃO

1.1 MOTIVAÇÃO

Suponhamos que se queira interligar um determinado número de cidades por meio de rodovias, e para diminuir os gastos com matéria-prima, o comprimento total da rede deve ser o menor possível; esse é um exemplo do *problema da árvore de Steiner* (ou STP, do inglês *Steiner tree problem*); um problema de otimização combinatória presente em diversas aplicações (DU; HU, 2008), como modelagem de estruturas biológicas (MONDAINI; OLIVEIRA, 2005), projetos de circuitos integrados VLSI (AREIBI et al., 2001), robótica (KRUPKE et al., 2015), construção e manutenção de sistemas, como os de distribuição de energia (HWANG et al., 1992), transporte, esgoto (HSIE et al., 2019), comunicação (MIGUEZ et al., 2002), etc.

Ao abordar o STP no espaço euclidiano, o problema passa a ser denominado *problema da árvore de Steiner Euclidiano* (ou ESTP, do inglês *Euclidean Steiner tree problem*). O objetivo do ESTP é conectar um conjunto de pontos no plano euclidiano, sendo permitida a adição de pontos extras denominados *pontos Steiner*, com a finalidade de se obter uma rede com o menor comprimento possível. O ESTP é um problema fácil de entender, porém difícil de resolver, pois a medida que a quantidade de pontos aumenta, a dimensão de possíveis soluções que devem ser verificadas cresce de forma exponencial (GILBERT; POLLAK, 1968), e como não se conhece uma solução ótima que pode ser encontrada por meio de um algoritmo de tempo polinomial para resolvê-lo, o ESTP discreto pertence à classe de problemas *NP-Difícil* (GAREY et al., 1977).

Devido a sua importância para a resolução de diversos problemas do mundo real, vários estudos propõem formas de resolver o ESTP por meio de algoritmos, entretanto, a maioria apresenta um alto consumo computacional a medida que a quantidade de pontos fixos aumenta, logo, a utilização de alternativas que computem boas soluções em tempo computacional aceitável se faz necessária.

1.2 OBJETIVO

Com o objetivo de obter soluções aproximadas de boa qualidade em tempo computacional viável para o ESTP no $\mathbb{R}^2$ (espaço bidimensional), este trabalho apresenta duas heurísticas híbridas baseadas em *algoritmos evolutivos* (ou EA, do inglês *evolutionary algorithm*), ou seja, métodos estocásticos que procuram a melhor solução para um problema dentre um grande número de soluções possíveis, e utilizam técnicas inspiradas na biologia populacional, genética e evolução (LUKE, 2013). A primeira heurística possui como base o algoritmo *evolução diferencial* (DE, do inglês *differential evolution*) (STORN; PRICE, 1997), e a segunda é baseada em um *algoritmo genético* (GA, do inglês *genetic algorithm*) (GOLDBERG, 1989). Espera-se que com a utilização de EAs híbridos, em vez de um EAs puros, seja alcançada uma melhora na velocidade de convergência do algoritmo e a obtenção de soluções com maior qualidade.

Nesse trabalho, o DE é utilizado para investigar estratégias de mutação. Essa heurística utiliza a triangulação de Delaunay para auxiliar na inserção de pontos Steiner em boas posições. A partir dos pontos, uma árvore geradora é construída, sua topologia é ajustada para se transformar em uma árvore de Steiner (ou ST, do inglês *Steiner tree*), e ao final, os pontos Steiner são posicionados em suas coordenadas ótimas. Ao utilizar uma estrutura geométrica para inicializar a população inicial, é esperado que o algoritmo melhore sua velocidade de convergência.

O GA permite investigar o efeito da sobreposição de gerações, o comportamento de operadores de cruzamento e mutação para árvores, e o impacto da aplicação de estratégias evolutivas. O algoritmo utiliza uma população de árvores de geradoras que são transformadas em árvores de Steiner. Os operadores genéticos são gulosos, pois tendem a favorecer arestas de menor custo, o que resulta na criação de topologias próximas à de árvores geradoras mínimas, que possivelmente podem ser transformadas em árvores de Steiner de boa qualidade.

1.3 ORGANIZAÇÃO DA DISSERTAÇÃO

Este trabalho está organizado da seguinte maneira: no Capítulo 2 é apresentado a história do problema; no Capítulo 3 são abordados alguns conceitos de teoria dos grafos; no Capítulo 4 é apresentado um problema relacionado, denominado problema da árvore geradora mínima; no Capítulo 5 são mostrados a definição e princípios que definem o ESTP; no Capítulo 6 os principais algoritmos criados para o ESTP são apresentados e discutidos; no Capítulo 7 é apresentado os fundamentos das meta-heurísticas utilizadas; no Capítulo 8 é explicado o funcionamento das heurísticas desenvolvidas; no Capítulo 9 são descritos os testes computacionais realizados e no Capítulo 10 são apresentadas as considerações finais.

2 A ORIGEM DO PROBLEMA DE STEINER

Para entender a origem do problema de Steiner, inicialmente é necessário compreender o problema Fermat-Torricelli, que é considerado o caso mais simples do problema, assim como as principais propriedades originadas do estudo da versão generalizada do problema de Fermat. As referências apresentadas nesse capítulo são fundamentadas nos estudos de Hwang et al. (1992) e Brazil et al. (2013).

2.1 O PROBLEMA FERMAT-TORRICELLI

Pierre de Fermat (1661–1665) descreveu tal problema por volta 1643, em seu trabalho “*Method for Determining Maxima and Minima and Tangents to Curved lines*”, em que era apresentado um problema que consistia em encontrar um ponto no plano cuja soma das distâncias a três pontos dados fosse mínima (FERMAT, 1891).

A primeira solução conhecida para este problema foi proposta pelo físico e matemático Evangelista Torricelli (1608–1647) (TORRICELLI, 1919), e é apresentada na Figura 1(a). Quase um século depois, no ano de 1750, Thomas Simpson (1710–1761) em seu livro “*Doctrine and Application of Fluxions*” reapresenta o problema por meio de um exercício (SIMPSON, 1750), e sua solução é mostrada na Figura 1(b).

Na Figura 1(a), a solução de Torricelli consiste em desenhar triângulos equiláteros do lado de fora do triângulo inicial formado pelos três pontos dados (v_1, v_2, v_3), e em seguida desenhar circunferências ao redor de cada um desses novos triângulos. O ponto de intersecção das circunferências (s_1) é a solução do problema e é chamado de *ponto de Torricelli* (TORRICELLI, 1919).

Na Figura 1(b), Simpson propõe uma solução que consiste em desenhar triângulos equiláteros do lado de fora do triângulo inicial (v_1, v_2, v_3), e em seguida desenhar retas a partir dos vértices exteriores dos triângulos equiláteros até os vértices opostos do triângulo inicial; a solução é o ponto de intersecção das linhas (s_1); o que ficou conhecido como *Linhas de Simpson*

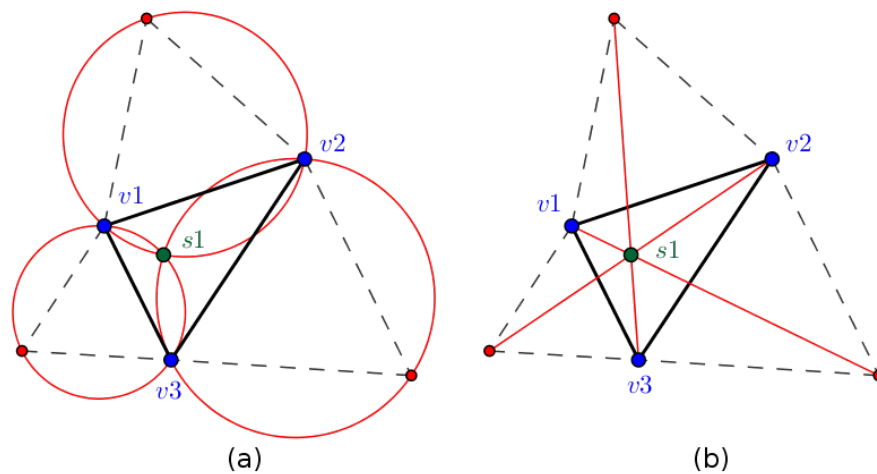


Figura 1: Ponto de Torriceli e linhas de Simpson

(SIMPSON, 1750).

Bonaventura Cavalieri (1598–1647) em seu livro “*Exercitationes Geometricae Sex*” de 1647 mostrou que os seguimentos de linhas que ligam os três pontos iniciais ao ponto de Torricelli possuem 120° uns com os outros. Quando um dos ângulos no triângulo dado é menor que 120° , o ponto de Torricelli fica fora do triângulo dado e não é mais o ponto de minimização. O ponto de minimização neste caso é o vértice do ângulo obtuso (CAVALIERI, 1647).

Em 1834 Heinen demonstrou que o comprimento de cada uma das linhas de Simpson é igual à soma das distâncias do ponto de Fermat aos três pontos dados (HEINEN, 1834).

2.2 O PROBLEMA DE FERMAT GENERALIZADO

Em 1811 o matemático francês Joseph Diaz Gergonne (1771–1859) publicou o primeiro trabalho que trata do problema de Steiner euclidiano, por meio de seu jornal intitulado “*Annales de mathématiques pures et appliquées*”. Em seu trabalho Gergonne pergunta como obter uma rede de comprimento mínimo interconectando um conjunto de pontos conhecidos no plano e discute resoluções para problemas com 4 a 6 pontos (GERGONNE, 1810). Esse novo problema proposto por Gergonne é diferente do problema Fermat-Torricelli para múltiplos pontos, pois deixa claro que a rede não fica restrita a adição de um único ponto extra. Na Figura 2 vemos as figuras desenhadas por Gergonne, em que os vértices A, B, C, D, E e F são os pontos iniciais, e os vértices M, N, P e Q são os pontos Steiner encontrados.

Após Gergonne muitos outros matemáticos contribuíram para o problema com diversos trabalhos. Por volta de 1836, Carl Friedrich Gauss (1777–1855) trocou diversas cartas com o astrônomo Heinrich Cristian Schumacher (1780–1850) discutindo o problema para quatro

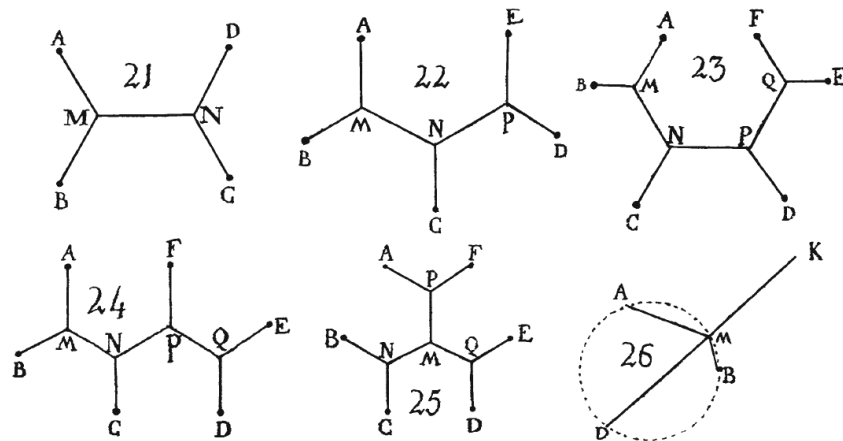


Figura 2: Figuras de Gergonne

Fonte: (GERGONNE, 1810)

pontos (BRAZIL et al., 2013). Em 1879, Karl Bopp (1856–1905) publicou a dissertação intitulada “*On the shortest connection system for four points*” em que propõe soluções geométricas para o problema com quatro pontos (BOPP, 1879). Em 1890, Edward Hoffmann (1858–1923) escreveu o trabalho intitulado “*On the shortest connection system for four points in the plane*” em que propõe que a solução para um problema com n pontos, será uma árvore com no máximo $n - 2$ vértices Steiner, e prova que para um problema de quatro pontos a melhor solução é uma árvore com dois pontos Steiner (HOFFMANN, 1890).

Em 1934, os matemáticos tchecos Vojtěch Jarník e Miloš Kössler publicaram um trabalho que trata do problema para espaços euclidianos n -dimensionais, e não somente no espaço plano. Eles concluíram que o ângulo de cada vértice é de pelo menos $2\pi/3$ e que cada vértice Steiner possui grau 3 (JARNÍK; KÖSSLER, 1934).

Algum tempo depois, em 1941, Richard Courant e Herbert Robbins lançaram o livro “*What is Mathematics?*”. A seção 5 do capítulo VII trazia como título “*Steiner’s Problem*” e a origem do problema era atribuída ao matemático Jakob Steiner (1796–1863) (COURANT; ROBBINS, 1941). Como o livro não era uma publicação matemática técnica, esse popularizou-se, e o problema acabou sendo chamado equivocadamente de “Problema da árvore mínima de Steiner”. Apesar de Steiner ter trabalhado no problema, não há contribuições com o tema para que este fosse batizado com seu nome (BRAZIL et al., 2013).

3 TEORIA DOS GRAFOS

Uma árvore é um grafo, logo, para que se possa apresentar e discutir formalmente o problema de Steiner, é necessário possuir uma compreensão básica da teoria dos grafos com foco nas estruturas e propriedades dos grafos e árvores. As principais referências desse capítulo são oriundas dos trabalhos de Even (2011), Jungnickel (2013) e Diestel (1997).

3.1 GRAFOS

Um grafo $G = (V, E)$ é uma estrutura formada por um conjunto de vértices (ou nós, ou pontos) $V = \{v_1, v_2, \dots\}$ e um conjunto de arestas (ou linhas) $E = \{e_1, e_2, \dots\}$ onde cada aresta conecta um par de vértices. Salvo indicação contrária, V e E são assumidos como finitos e nesse caso, G também é considerado finito. Um grafo também pode ser chamado de rede, portanto os termos são intercambiáveis (EVEN, 2011; JUNGnickel, 2013).

Na Figura 3 temos $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ e $E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7\}$. A aresta e_1 é *incidente* aos vértices v_1 e v_2 , pois os conecta. Os vértices v_1 e v_2 são os *pontos terminais* da aresta e_1 . No caso da aresta e_3 , os pontos terminais são o mesmo (v_3), logo essa aresta é denominada *laço*. As linhas e_6 e e_7 são chamadas de *paralelas*, pois possuem os mesmos vértices terminais (EVEN, 2011; JUNGnickel, 2013).

O *grau* de um vértice é o número de vezes que esse é utilizado como ponto terminal. O grau de um vértice v é denotado por $d(v)$. Um laço utiliza seu ponto terminal duas vezes. Na Figura 3, $d(v_1) = 1$, $d(v_2) = 3$, $d(v_3) = d(v_5) = 4$, $d(v_4) = 2$. Um vértice com grau zero é denominado *isolado*, como é o caso de v_6 em que $d(v_6) = 0$ (EVEN, 2011).

Para um grafo $G(V, E)$, em que $|E|$ representa o número de arestas e $|V|$ o número de vértices. O número de arestas em um grafo é igual à soma do grau de cada vértice dividido por dois, $|E| = \frac{\sum_{i=1}^n d(v_i)}{2}$ (DIESTEL, 1997).

Se a aresta e possui os vértices u e v como pontos terminais, é dito que e *conecta* u e v , ou que u e v são *adjacentes* (EVEN, 2011; JUNGnickel, 2013).

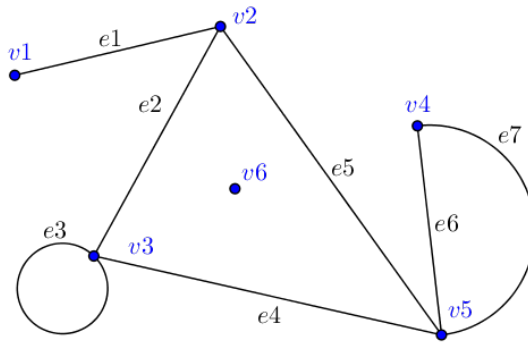


Figura 3: Grafo simples

Um *caminho* é uma sequência qualquer de arestas adjacentes que ligam dois vértices. Um caminho é *simple* se nenhum vértice aparece nele mais de uma vez. Considerando a Figura 3, as sequências $e1, e3, e3, e4$ e $e7, e4, e2$ são consideradas caminhos. O seguimento $e2, e3, e6$ não é considerado um caminho, visto que, para ser um, uma aresta e sua sucessora (e_n e e_{n+1}) devem possuir um vértice terminal em comum (DIESTEL, 1997).

Um *circuito* é um caminho que inicia e termina no mesmo vértice. Um circuito é *simple* se nenhum vértice, além do vértice de início/fim, aparece mais de uma vez, e o vértice de início/fim não aparece em nenhum outro lugar no circuito. Na Figura 3, $e2, e4, e5$ é um exemplo de circuito (JUNGNICKEL, 2013).

Um grafo $G(V, A)$ é considerado *conexo* se, para cada par de vértices, existir um caminho que os conecta. Caso contrário o grafo é tido como *desconexo* (DIESTEL, 1997). Na Figura 3 o grafo apresentado é desconexo, pois o vértice $v6$ está isolado.

Um grafo $G'(V', E')$ é chamado de *subgrafo* de um grafo $G(V, E)$ se $V' \subseteq V$ e $E' \subseteq E$, escrito como $G' \subseteq G$. De maneira menos formal, é dito que G' está contido em G (EVEN, 2011; DIESTEL, 1997). A Figura 4(b) mostra um subgrafo do grafo mostrado na Figura 4(a).

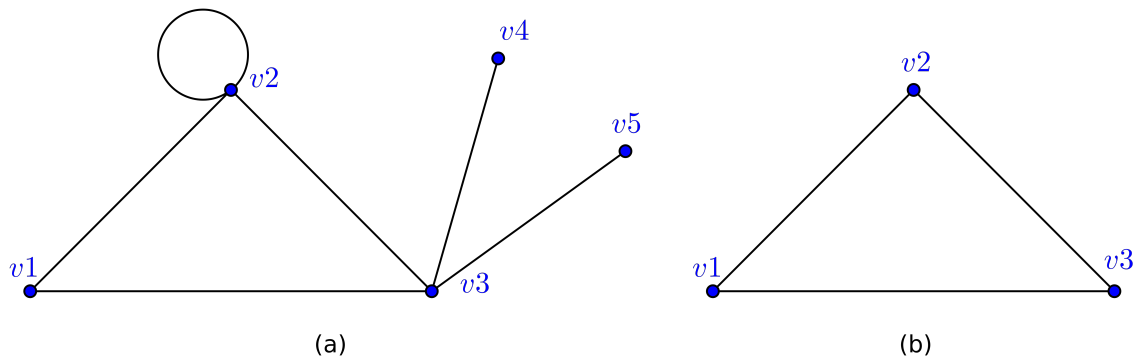


Figura 4: Subgrafo

O comprimento de uma aresta e é denotado por $l(e)$. Um grafo que possui o comprimento especificado em cada uma de suas arestas é denominado *ponderado* (EVEN, 2011). A Figura 5 apresenta um grafo ponderado, em que cada aresta possui um valor numérico que representa a distância entre seus vértices terminais.

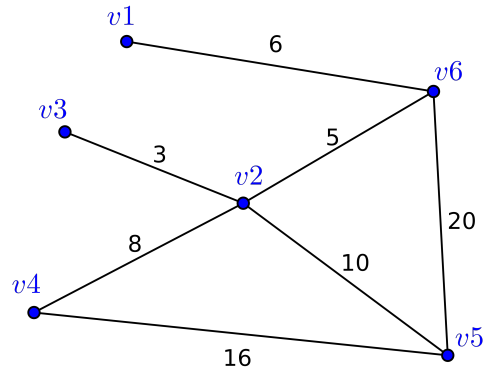


Figura 5: Grafo ponderado

3.2 ÁRVORES

Seja $G(V, E)$ um grafo não direcionado, finito ou não. É dito G um *circuito-livre* se não existir nenhum circuito simples em G . G é denominado *árvore* se este é conexo e é um circuito-livre. Logo, uma árvore nada mais é do que um grafo conexo sem circuitos (EVEN, 2011). O Teorema 1 define as condições para a existência de uma árvore (EVEN, 2011; DIESTEL, 1997).

Teorema 1. *As seguintes quatro condições são equivalentes:*

1. G é uma árvore.
2. G está livre de circuitos, mas se um novo vértice for adicionado a G , um circuito simples é Formado.
3. G não possui laços, e para cada dois vértices, existe um caminho simples único para conectá-los.
4. G está conectado, mas se qualquer aresta é eliminada, a conectividade de G é interrompida.

Outra forma comum de definir uma árvore finita é mostrada por meio do Teorema 2 (EVEN, 2011).

Teorema 2. Seja $G(V, E)$, um grafo finito e $n = |V|$. O seguinte é equivalente:

1. G é uma árvore.
2. G é livre de circuitos e possui $n - 1$ arestas.
3. G é conexo e possui $n - 1$ arestas.

Corolário 1. Uma árvore finita, com mais de um vértice tem pelo menos duas folhas.

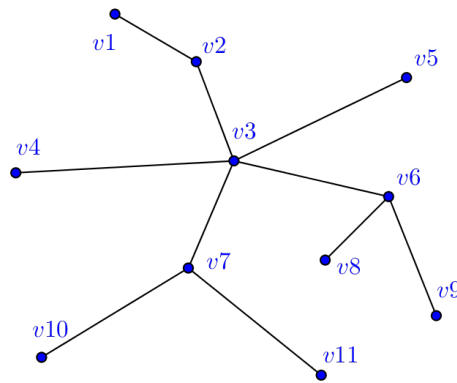


Figura 6: Árvore

A Figura 6 mostra um grafo conexo sem circuitos, ou seja, uma árvore. Em uma árvore um vértice que possui grau um é denominado *folha* (JUNGnickel, 2013). Na Figura 6 os vértices $v1, v4, v5, v8, v9, v10, v11$ são folhas.

3.3 GRAFOS EUCLIDIANOS

Um grafo euclidiano é um grafo em que os vértices são pontos no plano euclidiano e os comprimentos das arestas entre vértices são as distâncias euclidianas entre os pontos. Para calcular tal comprimento, faz-se necessário utilizar a *distância euclidiana*. Se $P = (p_1, p_2, \dots, p_n)$ e $Q = (q_1, q_2, \dots, q_n)$, então a distância euclidiana entre P e Q é dada pela Equação 1 (DEZA; DEZA, 2009).

$$d(P, Q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} = \sum_{i=1}^n (p_i - q_i)^2 \quad (1)$$

Neste trabalho o problema é abordado em um plano euclidiano bidimensional ($\mathbb{R}^2$), sendo assim, é considerado somente distâncias euclidianas onde $n = 2$, em que cada vértice

possui um valor x e y atribuído, de forma a representar sua posição no plano cartesiano. Dessa forma, a distância entre dois pontos $P = (p_x, p_y)$ e $Q = (q_x, q_y)$ é dada pela Equação 2 (DEZA; DEZA, 2009)

$$d(P, Q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2} \quad (2)$$

Seja L uma matriz de comprimentos entre vértices, com $l_{i,j}$ sendo a distância euclidiana entre os vértices v_i, v_j quando estes são conexos. O comprimento euclidiano total de T é dado pela Equação 3 (GILBERT; POLLAK, 1968).

$$\sum_{(i,j) \in T} W_{v_i, v_j} \quad (3)$$

A Figura 7 mostra um grafo com quatro vértices $v1 = (1, 1)$, $v2 = (2, 2)$, $v3 = (2.5, 1.5)$, $v4 = (3, 2)$ no plano euclidiano. O comprimento das arestas que interligam os vértices é obtido por meio da distância euclidiana, e o comprimento total do grafo é dado pela soma do comprimento das arestas.

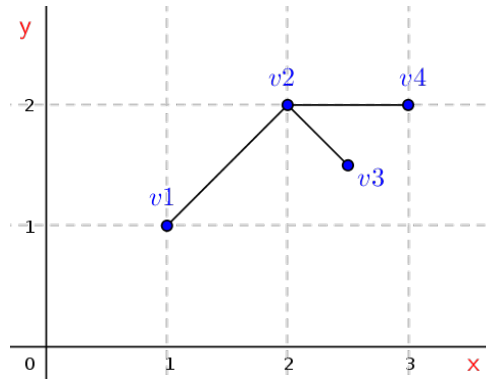


Figura 7: Grafo euclidiano

Na Figura 7, por exemplo, a distância entre os vértices adjacentes $v2$ e $v3$ é $d((2, 2), (2.5, 1.5)) = \sqrt{(2 - 2.5)^2 + (2 - 1.5)^2} = \sqrt{0.5}$.

4 ÁRVORES GERADORAS

A árvore geradora mínima (ou MST, do inglês *minimum spanning tree*) é um problema bastante conhecido na área de grafos e está profundamente ligado ao problema de Steiner. A MST é utilizada como limite superior para apurar a eficiência dos algoritmos propostos e nesse trabalho dois algoritmos empregados para computá-la inspiram o comportamento de componentes presentes nas heurísticas desenvolvidas.

4.1 ÁRVORE GERADORA MÍNIMA

Sendo $G(V, E)$ um grafo conexo finito em que cada aresta possui um comprimento conhecido, $l(e) > 0$. O problema da árvore geradora mínima consiste em encontrar um subgrafo conexo, $G'(V', E')$ em que a soma dos comprimentos das arestas $\sum_{e \in E'} l(e)$ é mínima (GILBERT; POLLAK, 1968). O grafo resultante acaba por ser sempre uma árvore, visto que G' é conexo e para favorecer a minimização do comprimento do caminho, não possui circuitos.

Um subgrafo de $G(V, E)$ que contém todos os vértices de G , $G'(V, E')$ e é uma árvore é chamado de *árvore de expansão* (DIESTEL, 1997), logo, a MST também é conhecido como *árvore de expansão mínima*.

De maneira menos formal, o problema da árvore geradora mínima consiste em encontrar a rede mais curta que interliga um conjunto de pontos fixos em um espaço métrico. A MST está profundamente ligado a árvore mínima de Steiner, servindo de base para que alguns algoritmos possam encontrá-la (CHANG, 1972; THOMPSON, 1973; BEASLEY, 1992; DREYER; OVERTON, 1998). Uma árvore mínima de Steiner sempre possui um comprimento menor que o de uma MST, logo, a MST serve de limite superior para o problema de Steiner (GILBERT; POLLAK, 1968). A Figura 8 mostra um exemplo de MST.

Na Figura 8 é mostrado um grafo conexo e ponderado com sete vértices ($n = 7$) e sua respectiva MST destacada por meio de linhas contínuas. Na MST não há circuitos, todo par de vértice está conectado e existe seis ($n - 1$) arestas conectando os vértices de forma que a

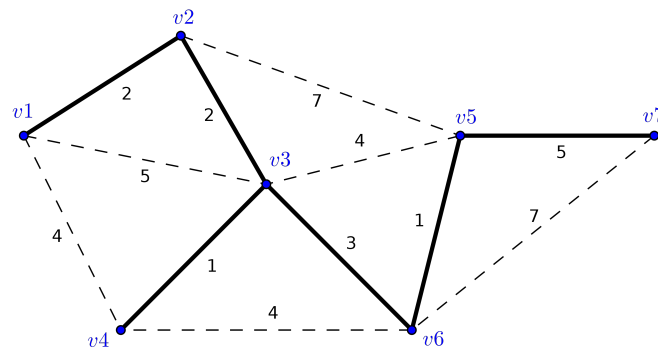


Figura 8: Árvore geradora mínima

distância total seja a mínima possível.

O primeiro algoritmo utilizado para resolver este problema foi proposto por Borůvka (1926). Os algoritmos mais populares foram criados por Kruskal (1956) e Prim (1957), sendo ambos algoritmos gulosos (ou gananciosos), pois a cada passo fazem uma escolha localmente ótima, ou seja, escolhem o melhor elemento para o momento (ERCIYES, 2018).

4.2 ALGORITMO DE KRUSKAL

O algoritmo de Kruskal (1956) cria uma solução adicionando a aresta de menor comprimento a cada passo. Inicialmente ele cria uma lista Q com as arestas de um grafo $G = (V, E, w)$ ordenadas pelo comprimento (custo) em ordem crescente. A partir da aresta de menor custo, adiciona as demais arestas na solução T , se não formarem ciclos com as arestas já contidas em T . O processo é concluído após todas as arestas serem processadas (KRUSKAL, 1956), conforme o Algoritmo 1.

Algoritmo 1: ALGORITMO DE KRUSKAL

Entrada: $G(V, E, w)$

Saída: MST T de G

```

1  $T \leftarrow \emptyset$ ;
2  $Q \leftarrow$  arestas de  $E$  ordenadas em ordem crescente pelo custo;
3 enquanto  $Q \neq \emptyset$  faça                                // Testa as arestas restantes
4   | seleciona uma aresta  $(u, v)$  de  $Q$ ;
5   | se  $(u, v)$  não faz um ciclo com vértices em  $T$  então
6   |   | adiciona  $(u, v)$  a  $T$ 
7   | fim
8 fim

```

Fonte: (ERCIYES, 2018)

4.3 ALGORITMO DE PRIM

O algoritmo de Prim (1957) constrói uma MST para um grafo $G(V, E, w)$ adicionando novos vértices a uma árvore em formação, ou seja, ele inicia escolhendo um vértice v qualquer e a partir desse cria uma árvore T . A cada iteração o algoritmo escolhe o vértice mais próximo de algum vértice de T para ser conectado a T . O algoritmo termina quando não existir mais vértices para serem adicionados em T (PRIM, 1957), e é mostrado no Algoritmo 2.

Algoritmo 2: ALGORITMO DE PRIM

Entrada: $G(V, E, w)$

Saída: MST $T(V, E_T)$ de G

```

1  $V_T \leftarrow \{s\};$ 
2  $T \leftarrow \emptyset;$ 
3 enquanto  $V_T \neq V$  faça           // visita os vértices restantes
4   |  selecione a aresta  $(u, v)$  com custo mínimo tal que  $u \in T$  e  $v \in G \setminus T;$ 
5   |   $V_T \leftarrow V_T \cup \{v\};$ 
6   |   $E_T \leftarrow E_T \cup \{(u, v)\};$ 
7 fim
```

Fonte: (ERCIYES, 2018)

5 ÁRVORES DE STEINER

Nesse capítulo é apresentado o problema da árvore de Steiner (STP), com foco principal no espaço euclidiano bidimensional ($\mathbb{R}^2$), logo, é abordado sua definição, principais propriedades, classes e variações do problema de Steiner.

5.1 DEFINIÇÕES

O problema da árvore mínima de Steiner (ou SMTP, do inglês *Steiner minimal tree problem*) apresenta a seguinte definição formal:

Encontrar uma rede geométrica $T = (V, E)$ (um grafo conexo com um conjunto de vértices V e um conjunto de arestas E) de modo que $N \subseteq V$ é um conjunto de pontos fixos e $S = V \setminus N$ é um conjunto (possivelmente vazio) de pontos conhecido como *pontos Steiner*, em que $\sum_{e \in E} l(e)$ (soma dos comprimentos das arestas $e \in E$) seja minimizado (BRAZIL et al., 2013).

Em outras palavras, esse problema consiste em encontrar uma rede para interligar um conjunto de pontos fixos, em um espaço métrico, com a possibilidade de adicionar novos pontos, de modo a minimizar a distância. Os vértices fixos são chamados de terminais e os novos vértices adicionados são conhecidos como pontos Steiner (HWANG et al., 1992).

Uma solução para o problema da árvore Steiner é considerada uma árvore T (caso contrário, não minimizará o comprimento), e pode assumir-se que qualquer ponto Steiner em T possui pelo menos três arestas incidentes.

Neste trabalho, o SMTP é tratado sob um espaço euclidiano bidimensional, logo, o problema passa a ser chamado *problema da árvore de Steiner euclidiano* (ESTP). Na Figura 9 são apresentados dois exemplos de ESTP.

A Figura 9(a) mostra uma ESTP com três vértices terminais v_1, v_2, v_3 , e um ponto Steiner s_1 localizado no *ponto de Torricelli* do triângulo formado pelos vértices terminais. A Figura 9(b) mostra uma ESTP com quatro vértices terminais v_1, v_2, v_3, v_4 e dois pontos Steiner

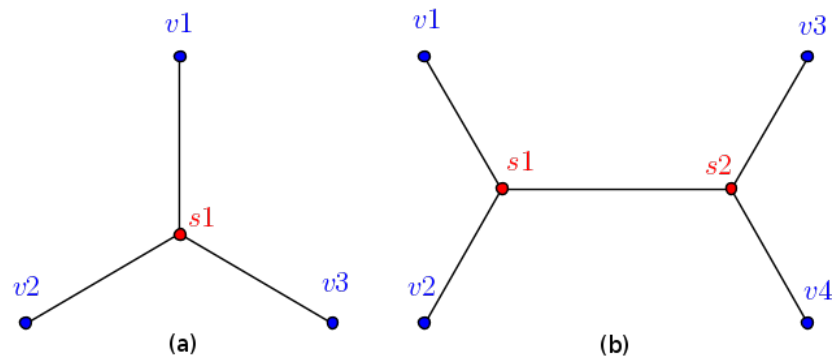


Figura 9: Árvore mínima de Steiner

$s1, s2$, conectados de forma a minimizar o comprimento total da rede.

Dados n pontos em um espaço métrico, para que se obtenha uma solução para o ESTP é necessário determinar:

1. A quantidade de pontos de Steiner que irão compor a solução.
2. As coordenadas dos pontos Steiner no espaço métrico.
3. A estrutura combinatória da solução, ou seja, uma representação das ligações entre os vértices (terminais e pontos de Steiner) da árvore encontrada.

O ESTP discreto é *NP*-Difícil (GAREY et al., 1977), portanto, não se sabe uma solução ótima que pode ser encontrada por um algoritmo de tempo polinomial.

5.2 TIPOS DE ÁRVORES

Árvores mínimas de Steiner são difíceis de encontrar porque uma árvore com comprimento *mínimo local* não é necessariamente *globalmente mínima*, não sendo o mínimo absoluto de todas as possíveis soluções (GILBERT; POLLAK, 1968).

A *topologia* de uma árvore T , é uma descrição das características topológicas de T , sendo a especificação de quais pares de vértices (terminais e pontos Steiner) estão conectados. A topologia apenas especifica as conexões, não o posicionamento dos pontos Steiner, portanto, não possui os comprimentos das arestas (HWANG et al., 1992). A Figura 10 mostra duas árvores topologicamente idênticas.

Na Figura 10 ambas as árvores possuem quatro vértices terminais $v1, v2, v3, v4$ e dois pontos Steiner $s1, s2$. Apesar de possuírem as mesmas conexões entre vértices, seus comprimentos não são iguais, visto que seus pontos Steiner estão em posições diferentes.

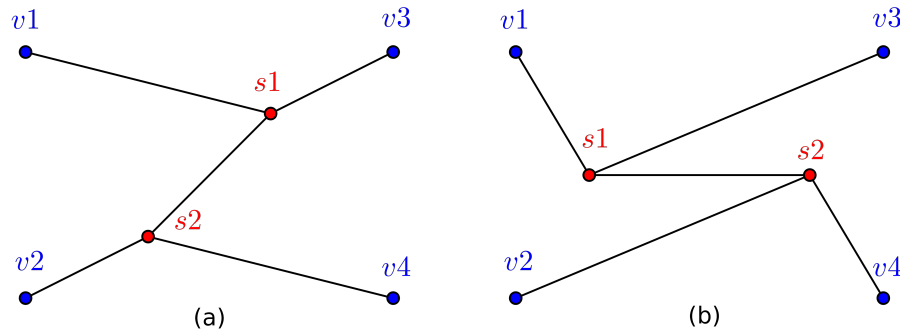


Figura 10: Topologias equivalentes

Uma árvore que é mais curta do que qualquer outra árvore com a mesma topologia é chamada de *árvore mínima relativa* para essa topologia (ou RMST, do inglês *relatively minimal Steiner tree*). Uma árvore mínima relativa não pode ter arestas com comprimento zero, ou seja, não deve possuir vértices sobrepostos, o que resulta no fato de algumas topologias não possuírem uma RMST.

Se uma ST com n pontos fixos, apresentar uma topologia com $s = n - 2$ pontos Steiner, considera-se que ela possui uma *topologia cheia*. Uma árvore é denominada *árvore de Steiner cheia* (FST, do inglês *full Steiner tree*) quando é a RMST para uma determinada topologia cheia (GILBERT; POLLAK, 1968).

Uma árvore é considerada *degenerada* se essa é obtida a partir do encolhimento de uma aresta de outra árvore com topologia diferente (GILBERT; POLLAK, 1968), como pode ser visto na Figura 11.

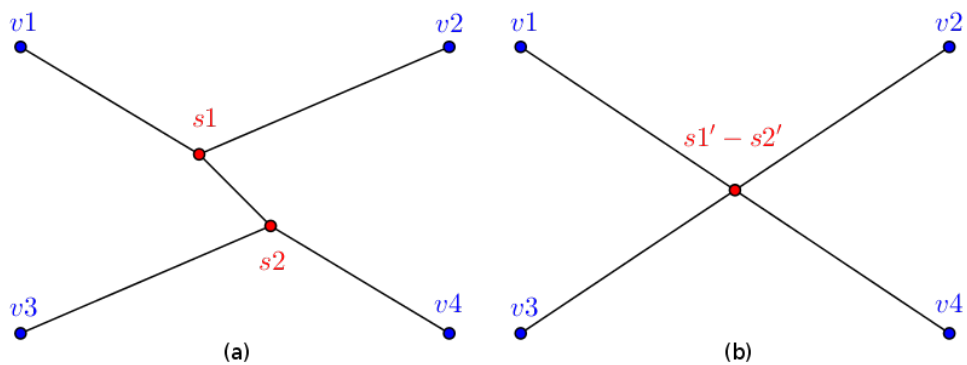


Figura 11: Topologia degenerada

Na Figura 11(a) é mostrada uma árvore com quatro vértices terminais $v1, v2, v3, v4$ e dois pontos Steiner $s1, s2$, e na Figura 11(b) é apresentada uma árvore degenerada, criada a partir do encurtamento do vértice que liga os dois pontos Steiner. O conjunto de degenerações de uma topologia G é denotado por $D(G)$.

Uma árvore mínima do Steiner (SMT, do inglês *Steiner minimal tree*) é uma ST com menor comprimento, assim como é uma árvore mínima relativa (RMST) para sua topologia (HWANG et al., 1992).

5.3 PROPRIEDADES

Nesta seção são apresentadas algumas propriedades básicas das STs que podem facilitar a busca por soluções ótimas. As principais referências utilizadas encontram-se nos trabalhos de Gilbert e Pollak (1968) e Hwang et al. (1992).

Se duas arestas se encontrarem em um ponto v formando um ângulo inferior a 120° , então pode se encurtar a árvore por meio da inserção de um ponto Steiner no ponto de Torricelli do triângulo formado pelos vértices terminais. Portanto, uma condição necessária para uma ST, é que duas arestas que se encontram não pode formar um ângulo menor que 120° . Essa propriedade é conhecida como *condição de ângulo*. Tal condição implica no fato de que nenhum vértice de uma ST pode ter mais do que três arestas. Outra implicação, é que não há linhas se cruzando em uma ST, visto que isso gera dois ângulos menores ou iguais a 90° (GILBERT; POLLAK, 1968). Tais propriedades são declaradas no Teorema 3.

Teorema 3. 1. *Em uma ST duas arestas sempre se encontram formando um ângulo maior ou igual a 120° .*

2. *Uma ST não possui arestas se cruzando.*

3. *Cada ponto Steiner de uma ST possui grau três.*

Por conta da condição de ângulo nenhum vértice em uma ST pode possuir mais de três arestas, logo, cada ponto Steiner possui exatamente três arestas formando três ângulos de 120° (GILBERT; POLLAK, 1968; HWANG et al., 1992), como pode ser visto na Figura 12.

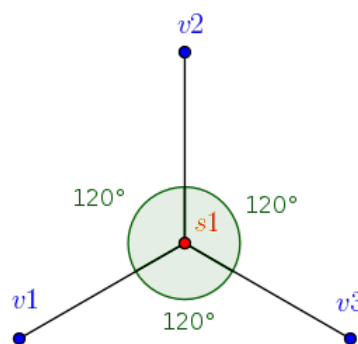


Figura 12: Condição de ângulo

Na Figura 12 é mostrado uma MST com três vértices terminais v_1, v_2, v_3 e um ponto Steiner s_1 , assim como os ângulos de 120° formados pelas arestas incidentes em s_1 .

Outra propriedade foi observada por Courant e Robbins (1941):

Teorema 4. *Uma ST com n vértices terminais contém no máximo $n - 2$ pontos Steiner.*

Demonstração. Suponha que uma ST tenha s pontos Steiner. Então possui $n + s - 1$ arestas. Uma vez que cada ponto Steiner tem três arestas e cada terminal possui pelo menos uma, o número de arestas deve ser pelo menos $(3s + n)/2$; A divisão em 2 é aplicada, pois cada aresta é contada com dois vértices. Segue que $n + s - 1 \geq (3s + n)/2$ ou $n - 2 \geq s$. $\square$

No plano euclidiano, uma forma é chamada de *convexa* se todos os pares de vértices conectados por uma aresta estão posicionados dentro da mesma. Dado um conjunto de pontos fixos no espaço euclidiano, um *casco convexo* é a forma convexa mínima que contém tais pontos. Em uma ST todos os pontos Steiner estão dentro de um *casco convexo* (GILBERT; POLLAK, 1968).

Uma ST com n vértices terminais e com $s = n - 2$ pontos Steiner é chamada de *árvore de Steiner cheia* (FST), visto que não há possibilidade de adicionar mais pontos Steiner. Uma FST possui pontos terminais com grau um e $n + s - 1 = n + n - 2 - 1 = 2n - 3$ arestas (HWANG et al., 1992).

Se uma topologia atender os requisitos de grau de uma ST, ela é considerada uma *topologia Steiner*, logo, pode gerar uma SMT. Considera-se que toda topologia de Steiner G pode ser obtida a partir de uma topologia de Steiner cheia (não necessariamente única) F . Sendo $D(F)$ o conjunto de todas as topologias degeneradas de F , $G \in D(F)$. Ao minimizar uma topologia de Steiner cheia F , a árvore obtida será a menor dentre as árvores em $D(F)$. Esta propriedade foi observada por Hwang e Weng (1992) e pode ser vista no Teorema 5.

Teorema 5. *Sendo F uma topologia de Steiner cheia, uma ST G cuja topologia está em $D(F)$ é a única árvore mínima entre as árvores cujas topologias estão em $D(F)$.*

Demonstração. Seja G uma árvore cuja topologia $T \in D(F)$, define-se $f_{ij} = 1$ se $[v_i, v_j]$ é uma aresta em G e $f_{ij} = 0$ caso contrário. Então o tamanho de T é: $|T| = \sum_{i < j} f_{ij} |v_i - v_j|$.

Considera-se $|T|$ como uma função das posições dos pontos Steiner. Uma vez que $|v_i - v_j|$ é uma norma, $|T|$ é estritamente convexo, exceto quando todas as arestas preservam suas direções em uma perturbação, então a convexidade não é necessariamente rigorosa. No entanto, mesmo quando as três arestas que incidem sobre um ponto de Steiner são paralelas,

movendo o ponto Steiner para o lado onde duas arestas incidentes se encontram, embora preservando as direções de todas as três arestas, diminui-se o comprimento total. Por isso, a convexidade rigorosa é garantida. Uma vez que uma ST é um mínimo local, a convexidade rigorosa garante que é também o único mínimo entre as árvores com topologias em $D(F)$. Logo, pode-se concluir que essa ST é a única com topologia pertencente a $D(T)$. $\square$

Por meio da quebra de algumas arestas, uma topologia de Steiner G pode ser dividida em subtopologias de Steiner cheias denominadas *componentes cheias* de G (GILBERT; POLLAK, 1968). Essa forma de decomposição pode ser vista na Figura 13.

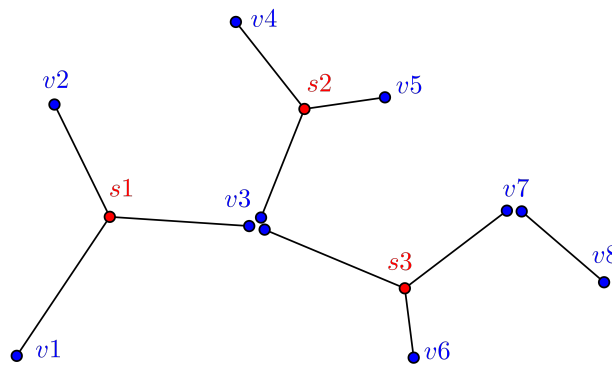


Figura 13: Decomposição de uma árvore

A Figura 13 mostra uma árvore com sete vértices terminais e três pontos Steiner sendo dividida em quatro componentes cheias por meio da quebra das arestas que se conectam aos vértices $v3$ e $v7$. O fato de uma topologia poder ser decomposta em subtopologias leva aos seguintes corolários:

Corolário 2. *Existe no máximo uma ST para uma dada topologia Steiner.*

Corolário 3. *Existe, no máximo, uma árvore relativamente mínima para uma dada topologia.*

5.4 RAZÃO DE STEINER

Dado um conjunto de pontos no espaço euclidiano V , uma árvore geradora mínima (MST) é a menor árvore que interconecta V utilizando somente arestas, enquanto uma árvore mínima de Steiner (SMT) é a menor árvore que interconecta V por meio de arestas e novos pontos que podem ser inseridos (denominados pontos Steiner), logo, uma SMT pode possuir um comprimento total menor ou no mínimo igual ao de uma MST, como pode ser visto na Figura 14.

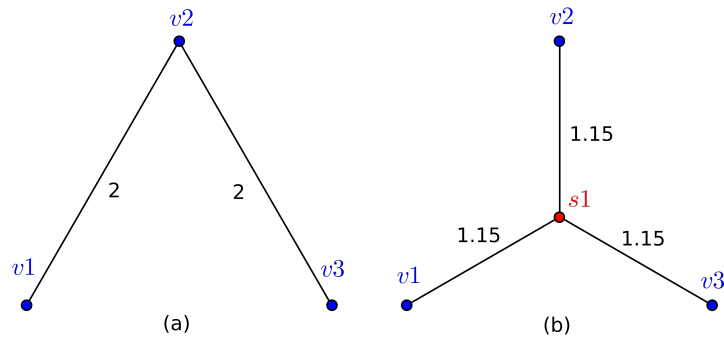


Figura 14: MST e SMT de um triângulo equilátero

A Figura 14(a) mostra a MST para um conjunto de vértices $v1, v2, v3$ com um comprimento total igual a 4, enquanto a Figura 14(b) mostra uma SMT para o mesmo conjunto de pontos com comprimento total igual a 3.45, logo, nesse caso a SMT se apresentou 13,75% menor que a MST. Sendo $|E|$ o comprimento total das arestas que interconectam o conjunto de vértices E , então para um conjunto N de pontos, é reconhecido que $|SMT(N)| \leq |MST(N)|$.

A razão de Steiner (SR, do inglês *Steiner ratio*) é a maior proporção de comprimento entre uma SMT e uma MST já encontrada para todos os problemas possíveis, sendo essa $r = \frac{|SMT|}{|MST|} = \frac{\sqrt{3}}{2}$. O valor da SR foi conjecturado por Gilbert e Pollak (1968), e mostra que $|MST|$ nunca excede $|SMT|$ em mais de $\frac{\sqrt{3}}{2} - 1 \simeq 15,5\%$.

5.5 VARIANTES DO PROBLEMA DE STEINER

No mundo real, novas regras, restrições, espaços métricos e dimensões podem ser incorporados ao problema de Steiner, gerando assim diversas variantes.

O projeto de circuitos eletrônicos VLSI pode requerer a utilização de um espaço métrico diferente do euclidiano, denominado “geometria do táxi”, que utiliza a *distância de Manhattan* (também chamada de L_1), ao invés da distância euclidiana, para calcular o comprimento da árvore. Esse caso origina o *problema de árvore de Steiner retilíneo* (RSTP, do inglês *rectilinear Steiner tree problem*), em que todos os pontos de Steiner estão dispostos numa estrutura denominada *grade subjacente* (também chamada de *grade planar* ou *grade de Hanan*), que é o conjunto de pontos criados pelas interseções de linhas horizontais e verticais que ligam os pontos terminais (HANAN, 1966).

Um projeto de engenharia que já possui um conjunto de pontos e conexões já estabelecidos e precisam de otimização estrutural, é um cenário compatível com o *problema de Steiner em redes* (NSTP, do inglês *network Steiner tree problem*), também conhecido como *problema de*

Steiner em grafos (STPG, do inglês *Steiner tree problem in graphs*). Dado um grafo $G(V, E)$ conectado, não dirigido, com comprimentos de vértices não negativos, e um subconjunto $W \subseteq V$, o STPG visa encontrar um subgrafo conectado $G'(V', E')$ de G , de modo que $W \subseteq V'$ e que $l(G')$ (comprimento de G') seja mínimo (KOCH; MARTIN, 1998).

Ao se pensar em melhorar uma rede *wireless* ou um circuito VLSI, muitas vezes é necessário minimizar o comprimento das maiores arestas de uma árvore de Steiner, e esse é um exemplo do *problema da árvore de Steiner com gargalo* (BSTP, do inglês *bottleneck Steiner tree problem*), também conhecido como *min-max Steiner tree problem* (WANG; DU, 2002). O BSTP pode ser considerado um problema duplo, pois é possível abordar um problema cuja restrição é colocada no número de pontos Steiner e o objetivo no comprimento da maior aresta, assim como outro problema cuja restrição é colocada no comprimento da aresta mais longa e o objetivo sobre a quantidade de pontos Steiner (DU; HU, 2008).

Esses problemas podem ser trabalhados em dimensões maiores que dois, o que resulta em uma complexidade computacional maior e no surgimento de propriedades próprias da dimensão abordada, o que também leva a criação de métodos de resolução diferentes dos utilizados para o mesmo problema no $\mathbb{R}^2$.

6 MÉTODOS DE RESOLUÇÃO

Diversos estudos presentes na literatura de teoria de grafos mostram algoritmos de aproximação, exatos e heurísticos para o ESTP. Esse capítulo apresenta alguns desses algoritmos, com maior foco nas pesquisas de Thompson (1973), Smith (1992) e Smith et al. (1981), visto que as contribuições disponibilizadas por esses estudos são utilizadas nas heurísticas propostas neste trabalho.

Segundo Hwang et al. (1992), o primeiro algoritmo exato para resolver o ESTP em $\mathbb{R}^2$ foi proposto por Melzak (1961), porém, o primeiro código computacional conhecido foi desenvolvido por Cockayne e Schiller (1972). Esses algoritmos não possuem bom desempenho e se destinam a resolução de instâncias pequenas.

Em 1997 foi apresentado o algoritmo exato “*Geosteiner*” (WINTER; ZACHARIASEN, 1997), que pode resolver instâncias com mais de mil pontos e é considerado o estado-da-arte para o ESTP em $\mathbb{R}^2$. Para o ESTP em $\mathbb{R}^d$ com $d > 2$, o estado-da-arte é o algoritmo exato “*Smith+*” (FAMPA; ANSTREICHER, 2008). Quanto aos algoritmos não-determinísticos, Larhoven e Ohlmann (2010) apresentam bons resultados em tempo computacional viável, e Yang (2006) mostra soluções de melhor qualidade, mas com tempo computacional elevado.

6.1 ALGORITMOS EXATOS

Melzak (1961) propôs o primeiro algoritmo exato para resolver o ESTP. Dado um conjunto de vértices terminais V , o algoritmo gera todas as FSTs para cada subconjunto $v' \subset V$. Em seguida o algoritmo seleciona a FST de menor custo para cada um desses subconjuntos e concatena as FSTs resultantes de todas as maneiras possíveis até encontrar a SMT. Vinte e cinco anos depois Hwang (1986) apresentou um algoritmo para a construção de Melzak (1961) que possuía melhor desempenho. O fato de ter que gerar um grande número de topologias, e possuir um engenhoso método para minimiza-las, faz com que esses algoritmos sejam funcionais somente para instâncias pequenas do ESTP.

Smith (1992) desenvolveu o primeiro algoritmo exato para resolver o ESTP em $\mathbb{R}$ com dimensão maior que 2. O algoritmo utiliza um método “*branch-and-bound*” para enumerar todas as possíveis topologias de FSTs e as armazena em uma estrutura denominada “vetor-topologia”. Para que uma SMT seja obtida a partir de um vetor-topologia, os pontos Steiner devem estar em posições ótimas, e para esse fim é utilizado um método iterativo, similar ao método Gauss-Seidel, em que a cada passo, as coordenadas dos pontos Steiner tendem à posição ótima. O algoritmo trata as arestas como um conjunto de molas ideais, e a cada iteração resolve um sistema de equações lineares, com a finalidade de minimizar a energia potencial do sistema de molas.

Sendo T uma topologia de Steiner com p pontos fixos, o método utilizado por Smith para reposicionar os pontos Steiner consiste em resolver em cada iteração i o seguinte sistema de $p - 2$ equações lineares:

$$\vec{X}_k^{(i+1)} = \frac{\sum_{j|(jk) \in A} \frac{\vec{X}_j^{(i+1)}}{|\vec{X}_j^i - \vec{X}_k^i|}}{\sum_{(jk) \in A, j \in S} \frac{1}{|\vec{X}_k^i - \vec{X}_j^i|}}, \quad p + 1 \leq k \leq 2p - 2 \quad (4)$$

Na Equação 4, o conjunto de pontos Steiner é representado por S , e o n -vetor $\vec{X}_k$ representa as coordenadas do k -ésimo ponto, podendo ser um ponto fixo se $1 \leq k \leq p$, ou um ponto Steiner se $p + 1 \leq k \leq 2p - 2$. Essa equação deve ser resolvida para todos os pontos Steiner $\vec{X}_k^{(i+1)}$ em suas n -dimensões, até que todos os ângulos de T sejam maiores ou iguais a 120° .

O método proposto por Smith (1992) representa cada aresta da árvore como uma mola ideal, de forma que a Equação 4 é utilizada para equilibrar a energia potencial do sistema a cada iteração, logo, para qualquer posição inicial definida para os pontos Steiner ($\vec{X}_k^0$), esse método converge para as únicas coordenadas ótimas dos pontos Steiner, de forma a minimizar o comprimento total da árvore.

Fampa e Anstreicher (2008) propuseram aprimoramentos no algoritmo de Smith (1992), dando origem ao algoritmo “*Smith+*”. Nesse algoritmo, a fase de enumeração das FSTs é melhorada por meio da obtenção de melhores candidatas, de forma a eliminar um grande número de topologias inválidas. A posição dos pontos Steiner para uma topologia é determinada por meio de uma abordagem que utiliza uma formulação cônica. Os resultados computacionais são comparados com os obtidos pelo algoritmo original de Smith (1992), e demonstram um ganho substancial na qualidade das soluções e tempo de execução, sobretudo em $\mathbb{R}^d$ para $2 \leq d \leq 5$.

(WINTER, 1985) desenvolveu o algoritmo exato “*Geosteiner*”, que em anos depois foi aperfeiçoado por Winter e Zachariasen (1997). O algoritmo enumera algumas FSTs para um conjunto de pontos terminais, e em seguida concatena as subárvores de forma a obter uma SMT. No processo de enumeração, é utilizado uma abordagem geométrica que evita a criação de FSTs que descumpram os requisitos estruturais necessários para formar uma SMT. Warme (1998) mostrou que o *Geosteiner* pode resolver problemas com mais de mil pontos de forma ótima. Desde seu lançamento como *software* aberto no ano 2000, diversos aprimoramentos foram implementados, e são apresentados no estudo de Juhl et al. (2018).

6.2 ALGORITMOS EMBASADOS EM ÁRVORES GERADORAS

Alguns algoritmos utilizam a MST como base para o desenvolvimento da SMT. Um dos primeiros a adotar essa abordagem foi Thompson (1973), que propôs um método que consiste em inserir pontos Steiner de forma sequencial em uma MST, até obter uma topologia de FST, que então tem seus pontos Steiner reposicionados para posições ótimas, resultando em uma RMST. Dreyer e Overton (1998) desenvolveram uma heurística baseada no método de Thompson (1973) que utiliza busca local para reposicionar os pontos Steiner após o processo de inserção dos mesmos.

Para inserir um ponto Steiner em uma MST, o algoritmo de Thompson (1973) busca por um ponto fixo v_1 conectado a dois outros pontos (fixos ou Steiner) v_2 e v_3 , e se o ângulo formado em v_1 pela intersecção das arestas for menor que 120° , um ponto Steiner é inserido no ponto de Torricelli do triângulo formado por v_1, v_2, v_3 ; em seguida v_1 é desconectado de v_2 e v_3 para serem conectados ao ponto Steiner.

O processo de inserir pontos Steiner em uma MST de forma a conectar conjuntos de três vértices e em seguida remover as arestas que formam ciclos é denominado *steinerizações locais* (HWANG et al., 1992) e é mostrado na Figura 15.

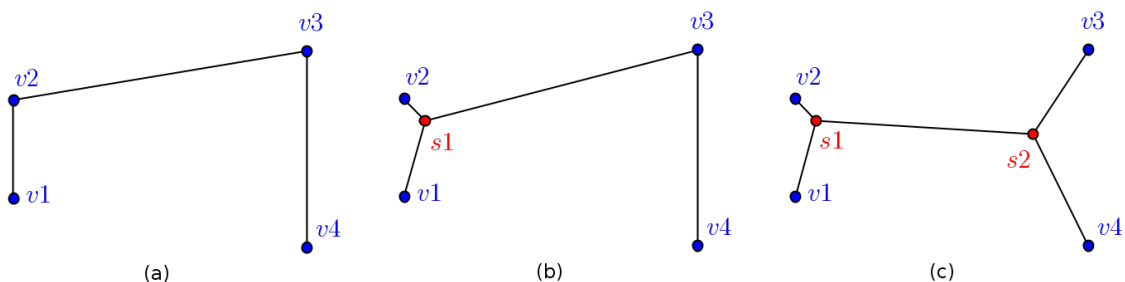


Figura 15: Processo de steinerizações locais

Na Figura 15(a) é mostrada uma árvore geradora. Por meio do vértice v_2 , que incide

em $v1$ e $v3$ é adicionado o ponto Steiner $s1$, que pode ser visto na Figura 15(b). Por meio do vértice $v3$ que é incidente em $s1$ e $v4$, é inserido o ponto Steiner $s2$, dando origem a uma árvore de Steiner, que é mostrada na Figura 15(c).

Aplicar steinerizações locais e reposicionar os pontos Steiner para suas posições ótimas é uma forma rápida de se conseguir uma árvore de Steiner, entretanto, nem sempre é possível obter o mínimo global para todas as instâncias do problema, pois nem todas as MSTs geram uma ST que é a melhor dentre as possíveis soluções. Laarhoven e Ohlmann (2010) mostram que para algumas instâncias esse método é falho, pois algumas MSTs podem acabar transformadas em RMSTs ao invés de SMTs, como pode ser visto na Figura 16.

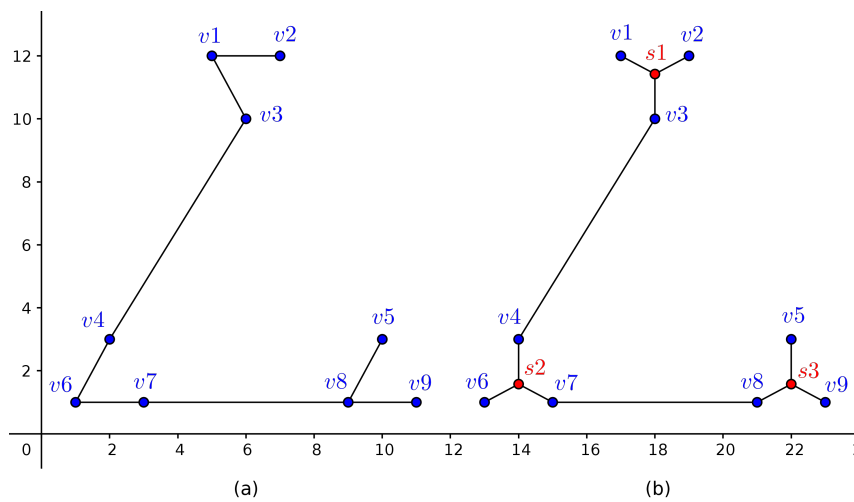


Figura 16: RMST gerada por steinerizações locais em uma MST

A Figura 16(a) mostra a MST de um problema com nove pontos, que após passar por steinerizações locais se transformou na ST mostrada na Figura 16(b). Apesar de ser uma boa solução, essa ST não é a melhor solução global para essa instância do ESTP. A solução considerada a SMT apresenta um ponto Steiner conectando $v3$, $v4$ e $v5$, de forma a unir as três subárvores formadas pelos triângulos menores e seus pontos Steiner, garantindo assim um comprimento total mínimo.

Beasley (1992) também utilizou um método de inserção de pontos Steiner em uma MST para encontrar soluções para o RSTP e ESTP. Essencialmente, essa heurística utiliza o método de Hwang (1986) para gerar uma FST para cada subgrafo conectado por quatro vértices na MST. Oliveira et al. (2014) propõem uma variante desse algoritmo que codifica as soluções por meio de uma estrutura denominada “nó-profundidade-grau”, de forma a garantir um tempo computacional menor que o apresentado pelo algoritmo original.

6.3 ALGORITMOS EMBASADOS NA TRIANGULAÇÃO DE DELAUNAY

Alguns estudos utilizam-se de estruturas geométricas para subdividir o espaço euclidiano, e assim inserir pontos Steiner em posições melhores, de forma a obter soluções iniciais de melhor qualidade para o ESTP.

Dado um conjunto de vértices $v_1, v_2, v_3, \dots$, o *diagrama de Voronoi* divide o espaço em regiões para cada vértice, sendo a região de cada vértice v definida por todos os pontos mais próximos de v do que para qualquer outro vértice. A *triangulação de Delaunay* é a única triangulação dos vértices dados em que a circunferência de qualquer triângulo não contém nenhum vértice em seu interior (FORTUNE, 1997). Essas duas estruturas são mostradas na Figura 17.

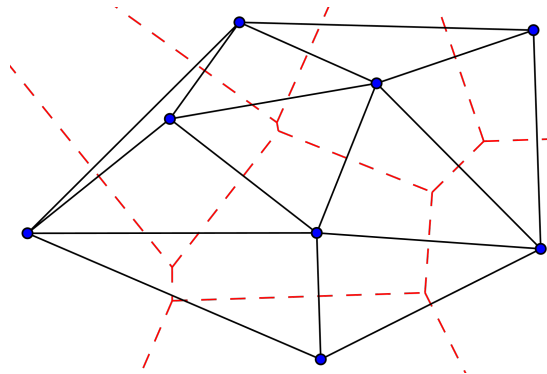


Figura 17: Diagrama de Voronoi e triangulação de Delaunay

A Figura 17 mostra o diagrama de Voronoi (linhas tracejadas), e a triangulação de Delaunay (linhas contínuas), para um conjunto de oito pontos no plano. Por meio dessa ilustração é possível observar que existe uma relação de dualidade e correspondência de um para um entre as duas estruturas (FORTUNE, 1997). Outra relação é encontrada entre a triangulação de Delaunay e a MST, pois uma MST é um subgrafo da triangulação de Delaunay, ou seja, as arestas que forma a MST para um conjunto de pontos estão contidas na triangulação de Delaunay desse conjunto (SHAMOS; HOEY, 1975). Como os triângulos formados por essa triangulação tendem a ser equiláteros, seus pontos de Torricelli são considerados um bom lugar para posicionar pontos Steiner.

O estudo de Smith et al. (1981) propõe uma abordagem de decomposição por meio da triangulação de Delaunay e do diagrama de Voronoi. Na primeira etapa do algoritmo (denominada fase de redução), o conjunto de pontos fixos é particionado em triângulos por meio da triangulação de Delaunay, e as soluções ótimas locais são encontradas se existirem. A segunda etapa do algoritmo (denominada fase de expansão) utiliza as propriedades do diagrama

de Voronoi e da MST do conjunto de pontos, e tem como objetivo criar uma ST por meio da concatenação dos subgrafos de cada triângulo.

Beasley e Goffnet (1994) utilizam um algoritmo baseado em “simulated annealing” que possui três fases: na primeira, denominada “expansão”, são inseridos pontos Steiner em triângulos criados por meio da triangulação de Delaunay; na segunda, intitulada “redução” é utilizada uma MST para remover pontos Steiner redundantes, e então, na fase de “re-expansão” são adicionados novos pontos Steiner caso seja necessário.

Para o ESTP em $\mathbb{R}^d$ para $2 \leq d \leq 5$, Laarhoven e Ohlmann (2010) propõem um algoritmo que utiliza a triangulação de Delaunay para gerar pontos Steiner candidatos para inserção, em seguida identifica os pontos Steiner que devem ser removidos por meio de uma MST, e então emprega programação cônica de segunda ordem para otimizar a posição dos pontos Steiner remanescentes.

6.4 META-HEURÍSTICAS

Lundy (1985) propõe um algoritmo baseado no método de busca estocástico “*Simulated Annealing*”. O algoritmo utiliza uma topologia de FST como solução inicial, e a cada geração, cria uma solução por meio de perturbações na topologia da melhor solução. Essas perturbações consistem na troca de arestas entre vértices, seguido de reposicionamento dos pontos Steiner de forma sequencial. Se a FST gerada possuir comprimento menor que o da melhor solução, essa FST passa a ser a melhor solução.

Forte et al. (2015) utilizam a meta-heurística “*Iterated local search*” para o ESTP em n dimensões. Essa abordagem emprega a estrutura vetor-topologia de Smith (1992) para armazenar uma FST, que inicialmente pode se gerada de forma gananciosa ou por meio de steinerizações locais em uma MST. Após a inicialização, é aplicado uma busca local sobre a solução inicial. De forma iterativa, a solução resultante tem sua estrutura alterada e em seguida passa por uma busca local. Se a solução gerada passar pelos critérios de aceitação, uma nova iteração ocorre. Como critérios de aceitação são propostos três procedimentos, que são: aceitar a melhor solução, aceitar uma solução que esteja entre as melhores e aceitar uma solução probabilisticamente.

Barreiros (2003) apresenta um algoritmo genético híbrido de duas fases. Na primeira fase decide como particionar um conjunto de vértices e na segunda encontra a SMT de cada partição. As soluções são codificadas por um vetor com as coordenadas dos pontos Steiner. Para definir uma topologia, é estabelecido que os pontos Steiner são conectados uns aos outros

com base na ordem do vetor, e os pontos fixos são conectados ao ponto Steiner mais próximo. Quanto aos operadores genéticos, o cruzamento se dá pela recombinação simples em um ponto, e a mutação se baseia em perturbações na posição ou na ordem dos pontos Steiner no vetor.

O algoritmo genético de Jesus et al. (2004) codifica cada solução por meio de um vetor com as coordenadas e os pontos adjacentes de cada ponto Steiner. Um operador de cruzamento uniforme e o operador de mutação tem a função de reposicionar pontos Steiner. Existe também um operador geométrico responsável por remover pontos Steiner muito próximos de outros pontos. Ao final os pontos Steiner com grau menor que três são removidos e os remanescentes são reposicionados por meio de um método iterativo até estarem no ponto de Torricelli do triângulo formado por seus vértices adjacentes.

Yang (2006) apresenta um algoritmo genético híbrido, em que cada solução armazena a posição de uma quantidade variável de pontos Steiner. Para efetuar a recombinação de soluções é utilizado um cruzamento aritmético, e para a mutação é empregado um operador que altera a posição de alguns pontos Steiner selecionados. A população inicial é criada por meio de steinerizações locais sobre uma MST para os pontos dados. Esse algoritmo possui mais dois operadores: um é responsável por remover pontos Steiner de grau um, visto que esses não ajudam na minimização do comprimento da árvore; e o outro move cada ponto Steiner s_1 conectado a três pontos v_1, v_2, v_3 para o ponto de Torricelli do triângulo formado por v_1, v_2, v_3 .

O estudo de Bereta (2018) emprega um algoritmo memético, que codifica a solução por meio de um vetor que armazena as coordenadas dos pontos Steiner, e um bit que informa se o gene (ponto Steiner) está ativo ou não. Dois procedimentos de busca local são responsáveis pela correção da topologia, e pelo ajuste da posição dos pontos Steiner para locais ótimos. As soluções modificadas são aceitas com probabilidade variada, de forma que se o valor for zero, ocorre o *efeito Baldwin* e se for um, então a evolução é do tipo *lamarckiana*.

7 COMPUTAÇÃO EVOLUTIVA

Nesse capítulo é abordado um conjunto de técnicas inspiradas pela biologia populacional, genética e evolução, que são coletivamente conhecidas como computação evolutiva (LUKE, 2013). Uma dessas técnicas é o “algoritmo evolutivo” (EA, do inglês *evolutionary algorithm*), no qual estão incluídos o “algoritmo genético” e a “evolução diferencial”, ambos utilizados no desenvolvimento das heurísticas propostas neste trabalho.

7.1 ALGORITMOS EVOLUTIVOS

Algoritmos evolutivos (EAs) são técnicas de otimização que simulam o processo de evolução natural. Por serem inspirados pela biologia, essas técnicas usam termos da genética e da evolução. Diversos EAs foram desenvolvidos desde a década de 70, sendo os mais comuns o *algoritmo genético* (GA) e as *estratégias de evolução* (ES, do inglês *evolution strategies*) (LUKE, 2013).

Essas técnicas são métodos baseados em população, pois mantêm um conjunto de soluções candidatas ao longo da execução do algoritmo. Essas soluções candidatas são chamadas de indivíduos, cromossomos ou vetor de parâmetros; e cada uma possui um valor de aptidão (ou *fitness*) associado, que define sua qualidade como solução. Os indivíduos promissores são escolhidos para reprodução, originando uma nova geração. No decorrer das iterações (gerações), os melhores indivíduos tendem a dominar a população, de forma a melhorar a qualidade da população.

Para melhor compreensão, são apresentados alguns dos principais termos originados da biologia que são utilizados por essa classe de algoritmos (LUKE, 2013):

- *Indivíduo, cromossomo ou vetor de parâmetros*: uma solução candidata.
- *Gene*: um componente em uma posição do cromossomo.
- *Aptidão*: qualidade de um indivíduo.

- *População*: um conjunto de indivíduos.
- *Seleção*: selecionar indivíduos com base em sua aptidão.
- *Mutação*: alterações aleatórias em alguns genes de um indivíduo.
- *Recombinação* ou *cruzamento*: criação de novos indivíduos com base em troca de material genético de dois indivíduos.
- *Genótipo* ou *genoma*: estrutura de dados de um indivíduo.
- *Fenótipo*: forma do indivíduo durante avaliação de aptidão.

A maioria dos EAs pode ser dividida em algoritmos *geracionais*, que atualizam todos os indivíduos de uma vez por iteração, e algoritmos de *estado estacionário*, que atualizam somente algumas soluções candidatas por geração (LUKE, 2013). Um EA geracional básico começa construindo uma população inicial, em seguida avalia a aptidão de todos os indivíduos da população, e com base na aptidão cria uma população. Em seguida é utilizado um operador para unir a geração de pais e de filhos para formar uma população que será utilizada na próxima geração. Esse ciclo continua até que algum critério de parada seja atingido. O Algoritmo 3 apresenta esse EA.

Algoritmo 3: ALGORITMO EVOLUTIVO GERACIONAL ABSTRATO

```

1  $P \leftarrow$  constrói população inicial;
2  $Melhor \leftarrow \emptyset$ ;
3 repita
4   AvaliaFitness( $P$ );
5   para cada indivíduo  $P_i \in P$  faça
6     se  $Melhor = \emptyset$  ou  $Aptid\tilde{a}o(P_i) > Aptid\tilde{a}o(Melhor)$  ent\~ao
7        $Melhor \leftarrow P_i$ ;
8     fim
9   fim
10   $P \leftarrow Une(P, Descendentes(P))$ ;
11 at\~e  $Melhor$  ser a solução ideal ou acabar o tempo;
12 retorna  $Melhor$ ;
```

Fonte: (LUKE, 2013)

7.2 EVOLUÇÃO DIFERENCIAL

Evolução Diferencial (DE) é um EA proposto por Storn e Price (1997), inicialmente para resolver o problema de ajuste polinomial de Chebychev, mas que pode ser utilizado para solucionar problemas de várias naturezas. O DE é mais comumente empregado na otimização de problemas contínuos modelados por números reais e como é um método baseado em população, utiliza-se de operadores de recombinação e mutação para criar indivíduos. Embora tenha uma operação simples e seja fácil de implementar e controlar, o DE apresenta uma convergência rápida quando comparado a outras heurísticas (STORN; PRICE, 1997).

A população inicial de um DE é composta de n indivíduos, e cada indivíduo é representado por um vetor de parâmetros X , cujo tamanho d não muda durante g gerações. Caso não exista nenhum conhecimento sobre o problema a ser resolvido, os vetores de parâmetros podem ser definidos por meio de uma distribuição de probabilidade uniforme.

Depois que a população é inicializada, cada vetor tem sua aptidão calculada. Então cada *vetor alvo* é manipulado por mutação e cruzamento gerando um *vetor experimental* que o substitui se for mais apto. Para controlar a pesquisa, definimos inicialmente valores para os parâmetros F , que controlam a mutação, Cr que controla o cruzamento e n que representa a quantidade de vetores da população.

Para cada *vetor alvo* X_i^g , onde i é o índice de vetor na população e g é a geração atual, três indivíduos diferentes (X_a^g, X_b^g, X_c^g) são escolhidos aleatoriamente para gerar um *vetor experimental* (X_i^{g+1}) por meio da Equação 5.

$$X_i^{g+1} = X_a^g + F(X_b^g - X_c^g) \quad (5)$$

Na Equação 5 a, b, c são inteiros no intervalo $[0, n - 1]$, diferentes um do outro e de i , portanto um DE deve ter uma população com pelo menos quatro indivíduos. O parâmetro F é um fator real e constante no intervalo $[0, 2]$, que controla a ampliação da variação diferencial $((X_b^g - X_c^g))$ (STORN; PRICE, 1997).

Após a mutação, alguns componentes j de cada *vetor alvo* (X_i^{g+1}) são substituídos por componentes do *vetor experimental* (X_i^g) por meio do operador de cruzamento, definido pela Equação 6. Nesse operador, um número real no intervalo $[0, 1]$ denotado por Cr representa a taxa de cruzamento.

$$X_{i,j}^{g+1} = \begin{cases} X_{i,j}^{g+1}, & \text{Se } rand() \leq Cr \\ X_{i,j}^g, & \text{Senão} \end{cases} \quad (6)$$

Na Equação 6, $rand()$ é um número real aleatório no intervalo $[0, 1]$, i é o índice do indivíduo na população, j é o índice do componente no i -ésimo vetor e g representa a geração. Nesse operador, quanto maior o valor de Cr , maior a tendência de componentes do *vetor alvo* serem adicionados ao *vetor experimental*.

Ao final, cada *vetor experimental* tem sua aptidão calculada, e se for melhor que seu respectivo *vetor alvo*, ele o substitui na população, de modo que uma evolução natural na qualidade das soluções ocorra. O processo pode ser finalizado quando o algoritmo atinge a convergência, é executado por um certo número de gerações ou período.

7.3 ALGORITMO GENÉTICO

Algoritmos genéticos (GAs) são procedimentos de busca estocástica inspirados nos princípios darwinianos da seleção natural e nas ideias da genética mendeliana. Um GA começa com uma população de soluções candidatas criadas de forma aleatória quando não há conhecimento sobre o problema e implementa exploração probabilística e paralela no espaço de busca usando operadores genéticos de seleção, cruzamento e mutação (GOLDBERG, 1989; MITCHELL, 1998).

Em um GA, a população inicial é composta de n indivíduos, e cada indivíduo é representado por um cromossomo X , com d genes. Após criada a população inicial, todos os indivíduos têm sua aptidão calculada, e no decorrer das gerações, indivíduos são selecionados para reprodução e mutação, gerando novos indivíduos que substituem seus progenitores. Após um determinado número de iterações, os indivíduos mais aptos tendem a dominar a população, resultando em uma melhora na qualidade das soluções. O algoritmo encerra ao atingir um determinado critério de parada (MITCHELL, 1998).

As soluções que compõem a população de um GA devem ser codificadas geneticamente, ou seja, representadas por meio de cromossomos. Geralmente um cromossomo pode ser representado por uma estrutura de dados do tipo vetor, e seus genes podem ser codificados por meio de números binários, inteiros ou reais. A escolha da codificação depende de como o problema é abordado, por exemplo, se o objetivo é armazenar as coordenadas dos pontos Steiner utiliza-se uma codificação por números reais, já se a intenção é saber quais pontos Steiner fazem parte de uma árvore (solução), uma codificação binária pode ser empregada para representar o

ponto Steiner (gene) como ativo (1) ou inativo (0).

Para abordar o ESTP por meio de um EA, é necessário utilizar representações (estruturas de dados) próprias para grafos, visto que as formas de codificação convencionais não são adequadas para representar topologias. Essas formas de codificação são consideradas diretas quando um grafo é representado por um conjunto de arestas, e os operadores genéticos são aplicados diretamente nessa estrutura; e indiretas quando o grafo (fenótipo) é construído com base nos dados de um cromossomo (genótipo), ou seja, os operadores genéticos são aplicados no cromossomo e o grafo é gerado por meio de um mapeamento genótipo-fenótipo (ROTHLAUF; GOLDBERG, 2006).

São exemplos de codificação direta para árvores as seguintes representações: matriz de adjacências, lista de adjacências, vetor de características (PALMER; KERSHENBAUM, 1995), conjunto de arestas (RAIDL; JULSTROM, 2003) e *NetKeys* (ROTHLAUF; GOLDBERG, 2006). Já para representar indiretamente uma árvore é possível utilizar as seguintes representações: número de *Prüfer* (ZHOU; GEN, 1997), *Blob Code* (JULSTROM, 2005), Nó-Profundidade (DELBEM et al., 2004) e Nó-Profundidade e Grau (DELBEM et al., 2012), entre outros.

O operador de seleção tem o objetivo de selecionar os indivíduos aptos para reprodução, e os métodos mais comuns são torneio e roleta. Em uma seleção por torneio, são escolhidos k indivíduos, e dentre esses é selecionado o indivíduo mais apto para reprodução. A seleção por roleta estabelece uma proporcionalidade entre a hipótese de ser escolhido com o valor de aptidão de cada indivíduo, ou seja, quanto maior a aptidão de um indivíduo, maior é possibilidade de ser selecionado para reprodução.

O operador de cruzamento tem a função de combinar os genes de dois indivíduos, gerando um indivíduo descendente que herde características dos dois progenitores, para então os substituir na população. O operador de mutação tem como objetivo (re)introduzir informação genética na população. Para fornecer alta localidade, um operador de mutação geralmente deve fazer uma pequena alteração em um indivíduo pai (GOLDBERG, 1989). Cada codificação permite diversos operadores de cruzamento e mutação, portanto essa escolha varia conforme a abordagem do problema.

O controle de um GA depende de alguns parâmetros que devem ser previamente definidos, logo, é estabelecido que a população terá n indivíduos, a probabilidade de ocorrência de cruzamento é representada pela taxa de cruzamento *Cruz* e a probabilidade de um indivíduo ser mutado é dada pela taxa de mutação *mut*. O critério de parada normalmente é definido como a execução por uma determinada quantidade de iterações, período ou quando o processo

evolutivo convergir, ou estagnar.

Em um GA de *estado estacionário*, somente alguns indivíduos são atualizados a cada geração, ao contrário da estratégia *geracional*, em que toda a população é atualizada. Essa abordagem foi popularizada pelo sistema “*G Enitor*” de Whitley (1988). Quando comparado com um GA geracional, o GA de estado estacionário utiliza menos memória e processamento por geração, pois há somente uma população por iteração e apenas uma pequena parcela da população é selecionada para reprodução. Como os indivíduos pais conseguem permanecer na população por um longo tempo, indivíduos aptos podem tomar conta da população rapidamente, resultado em uma convergência prematura (LUKE, 2013). O Algoritmo 4 mostra um GA de estado estacionário.

Algoritmo 4: Algoritmo genético de estado estacionário

```

1   $popTam \leftarrow$  tamanho da população;
2   $totGer \leftarrow$  quantidade de gerações;
3   $P \leftarrow \{\}$ ;
4  para  $popTam$  vezes faça
5       $novoRndInd \leftarrow$  {novo indivíduo aleatório};
6       $calculaAptidao(novoRndInd)$ ;
7       $P \leftarrow P \cup \{novoRndInd\}$ ;
8  fim
9   $Melhor \leftarrow getMelhorIndividuo(P)$ ;
10 repita
11      $Pai P_1 \leftarrow selecionaParaSubstituicao(P)$ ;
12      $Pai P_2 \leftarrow selecionaParaSubstituicao(P)$ ;
13      $filho C_1 \leftarrow cruzamento(P_1, P_2)$ ;
14      $C_1 \leftarrow mutacao(C_1)$ ;
15      $calculaAptidao(C_1)$ ;
16     se  $aptidao(C_1) < aptidao(Melhor)$  então
17          $Melhor \leftarrow C_1$ ;
18     fim
19      $Individuo P_d \leftarrow selecionaParaEliminacao(P)$ ;
20      $P \leftarrow P \cup \{C_1\}$ ;
21      $P \leftarrow P - \{P_d\}$ ;
22 até (executa totGer vezes) ou (Melhor é a solução ideal);
23 retorna  $Melhor$ ;

```

8 ALGORITMOS DESENVOLVIDOS

Para resolver o ESTP, é necessário obter a topologia, a quantidade e a posição dos pontos Steiner, logo, diversas abordagens podem ser empregadas para essa finalidade.

Para definir a quantidade ideal de pontos Steiner, é possível adicionar e remover pontos Steiner de topologias durante a execução, ou gerar FSTs e ao final retirar os pontos Steiner inviáveis (que estão acima ou muito próximos de outros pontos). Quando se trabalha com FSTs evita-se a busca pela quantidade de pontos durante a execução do algoritmo, porém o consumo de memória e processamento é maior, visto que as árvores terão o número máximo de pontos Steiner em todas as gerações.

Quanto a definição da posição ideal para pontos Steiner, pode-se manipular sua coordenada e então gerar a topologia em função dos pontos, ou colocar os pontos no lugar correto a partir da topologia. Gerar a topologia com base na posição dos pontos, por meio de algoritmos gulosos como Prim (1957) ou Kruskal (1956) pode resultar em pontos Steiner com grau diferente de três, logo, é necessário adicionar um algoritmo para corrigir a topologia, pois pontos Steiner que não possuem três arestas incidentes violam a propriedade de ângulo. Em contrapartida, obter a posição dos pontos Steiner com base na topologia de Steiner, é eficaz e rápido caso se utilize o algoritmo de Smith (1992) que se baseia em um modelo físico em que cada aresta é considerada uma mola ideal.

No que diz respeito à topologia, a dificuldade em se trabalhar diretamente com a topologia de Steiner está em criar operadores que consigam lidar corretamente com as árvores de Steiner de forma a não gerar ciclos, desconexões, ou infringir as restrições de grau dos pontos.

Com o objetivo de testar métodos diferentes de abordar o problema, esse capítulo apresenta dois algoritmos híbridos para o ESTP no $\mathbb{R}^2$. O primeiro se baseia em evolução diferencial e aborda o problema em espaço contínuo, ou seja, manipula a posição dos pontos Steiner e cria a topologia em função dos pontos. O segundo se embasa em um algoritmo genético e trata o problema como discreto, pois manipula topologias para criar árvores de Steiner.

8.1 EVOLUÇÃO DIFERENCIAL HÍBRIDO

Em um DE, a quantidade de componentes de um vetor é fixa durante todo o processo de busca, logo, é conveniente trabalhar com árvores de Steiner cheias (FSTs) e ao final utilizar um algoritmo para remover os pontos Steiner inviáveis, ou seja, pontos Steiner que estão acima ou muito próximos de outros pontos Steiner.

Para trabalhar o ESTP como um problema de natureza contínua, no DE cada componente de um vetor representa a coordenada (x e y) de um ponto Steiner no plano euclidiano. Estabelecido um conjunto de pontos fixos e pontos Steiner, algoritmos são utilizados de forma sequencial para criar uma FST, armazenar sua estrutura em uma matriz de adjacências e calcular seu comprimento, que é considerado a aptidão da solução.

Para inicializar a população é utilizada a triangulação de Delaunay sob os pontos fixos, e para isso é empregado um *sweep-line algorithm* simples baseado nos trabalhos de Biniaz e Dastghaibifard (2012) e Sinclair (2010). Após construir a triangulação de Delaunay para n pontos fixos, $n - 2$ triângulos são escolhidos aleatoriamente e para cada triângulo um ponto Steiner é inserido em seu ponto de Torricelli.

A partir de um conjunto de pontos fixos e pontos Steiner dispostos no plano, o algoritmo de Kruskal (1956) é utilizado para construir uma árvore. Como um ponto Steiner deve possuir grau três para satisfazer sua propriedade de ângulo, é adicionada uma restrição ao algoritmo de forma a não permitir que sejam adicionadas novas arestas a um ponto Steiner que possuir três arestas incidentes. Ao realizar esse processo, alguns pontos Steiner podem acabar ficando com grau menor que três, portanto devem ser removidos, pois não contribuem para a minimização da distância. Para remover um ponto Steiner de grau um, basta remover o ponto e a aresta que nele incide, já para remover um ponto Steiner v_1 conectado a dois pontos v_2 e v_3 , remove-se v_1 e suas arestas incidentes, e adiciona uma nova aresta conectando v_2 a v_3 .

Após remover os pontos Steiner inviáveis, é necessário adicionar novos pontos Steiner para corrigir a topologia da FST, e para isso é utilizado o método de Thompson (1973). Em uma topologia de FST, cada ponto fixo possui grau um, logo, é necessário corrigir todos os pontos fixos que possuem grau maior que um. Sendo v_1 um ponto fixo conectado a dois pontos v_2 e v_3 , aplica-se uma *steinerização local* em v_1 para inserir um novo ponto Steiner na árvore. Esse processo é realizado até que todos os pontos fixos da árvore possuam grau um, resultando em uma topologia de FST válida. Para que se tenha uma FST os pontos Steiner devem estar em posições ótimas e para essa isso é utilizado o algoritmo de Smith (1992).

A Figura 18 apresenta o processo de construção de uma árvore de Steiner a partir da

triangulação de Delaunay para a sexta instância do problema de 10 pontos da biblioteca de *benchmark OR-Library* (BEASLEY, 1990).

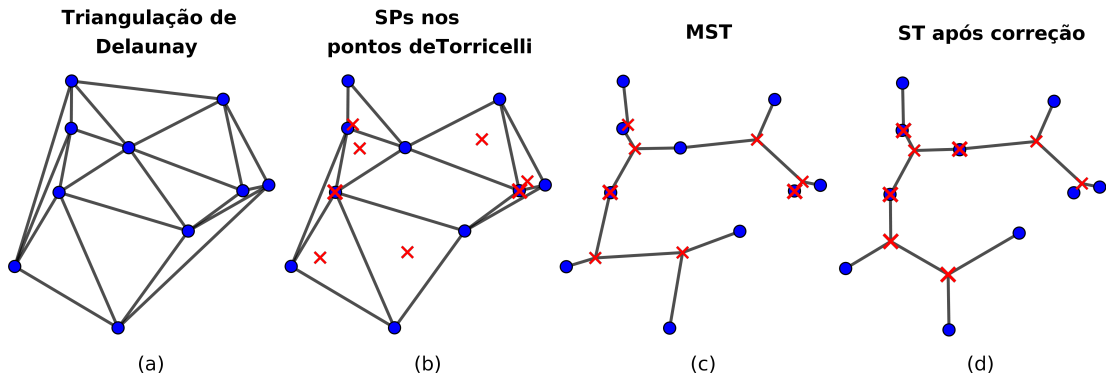


Figura 18: Construção da árvore de Steiner no DE

Na Figura 18(a) é mostrado 13 triângulos sob os pontos fixos, gerados por meio da triangulação de Delaunay. Como a instância em questão apresenta $n = 10$ pontos fixos, para se criar uma FST é necessário inserir $n - 2$ pontos Steiner, logo, na Figura 18(b), são escolhidos aleatoriamente 8 triângulos para que sejam adicionados pontos Steiner em seus pontos de Torricelli. Na Figura 18(c) é aplicado o algoritmo de Kruskal (1956) para gerar uma MST com restrição de grau, de forma que os pontos Steiner tenham no máximo grau três. A Figura 18(d) apresenta uma FST após o processo de correção de topologia, que remove pontos Steiner com grau menor que três, os reinsere em lugares adequados por meio do método de Thompson (1973) e corrige suas posições por meio do algoritmo de Smith (1992).

Após a inicialização da população, os operadores do DE modificam a posição dos pontos Steiner e uma nova topologia é criada por meio da construção da MST (Figura 18(c)), e corrigida por meio da remoção, reinserção e reposicionamento de pontos Steiner (Figura 18(d)). Assim que o DE finaliza as iterações, uma FST é retornada como melhor solução, e caso exista pontos Steiner posicionados sobre outros pontos, eles podem ser removidos, pois não contribuem para a minimização da topologia. Para eliminar um ponto Steiner $s1$ que ocupe a mesma posição de outro ponto denominado $v1$, basta conectar em $v1$ às arestas que incidem em $s1$ e em seguida eliminar $s1$.

8.2 ALGORITMO GENÉTICO HÍBRIDO

Para trabalhar o ESTP como um problema discreto, é utilizado um GA que representa as soluções por meio de árvores geradoras, as transforma em árvores de Steiner cheia (FSTs), reposiciona os pontos Steiner por meio da topologia, e ao final, retira os pontos Steiner inválidos

da melhor FST. Os operadores genéticos atuam sob a topologia das árvores geradoras, e como essas representam somente os pontos fixos, não há que se preocupar com restrições de grau dos pontos.

Para transformar uma árvore geradora em uma árvore de Steiner, é necessário aplicar steinerização local sobre todos os vértices fixos com grau maior um, até todos os vértices da árvore tenham grau um. Se um vértice $v1$ possuir grau maior que dois, deve-se iniciar o processo de steinerização pelo par de vértices adjacente a $v1$ que resultar no menor ângulo em $v1$. Esse processo permite que ao final das steinerizações de $v1$, o par de vértices adjacentes a $v1$ com maior ângulo em $v1$ ficará conectado a este.

Após todos os vértices fixos de uma árvore geradora passar pelo processo de steinerização local, é necessário reposicionar os pontos Steiner para que sua propriedade de ângulo seja atendida e assim a árvore tenha um comprimento total menor. Caso um ponto Steiner tenha como adjacentes somente pontos fixos, sua posição só precisa ser ajustada uma vez. Se um ponto Steiner incide em outro ponto Steiner, é necessário ajustar sua posição por diversas vezes, pois ao mudar a posição de um, o Steiner incidente terá sua propriedade de ângulo alterada e consequentemente não estará mais em sua posição ideal. Logo, é empregado o método iterativo de Smith (1992) para reposicionar os pontos Steiner para suas posições ótimas.

Assim que o GA finaliza as iterações, uma FST é retornada como melhor solução e caso haja pontos Steiner posicionados sobre outros pontos eles podem ser removidos, pois não contribuem para a minimização da topologia. Para retirar um ponto Steiner $s1$ posicionado em cima de outro ponto denominado $v1$, basta conectar os vértices incidentes em $s1$ em $v1$ e em seguida remover $s1$.

O GA proposto tem como objetivo específico a criação e modificação de árvores geradoras e para que essa finalidade seja alcançada de maneira eficiente a codificação dos cromossomos e os operadores genéticos devem ser próprios para a estrutura de dados árvore. Com base nisso, o algoritmo apresenta as seguintes características:

Representação de Soluções: O presente GA representa as soluções por meio de uma codificação direta por matriz de adjacências e dessa maneira os operadores genéticos são aplicados diretamente sobre a estrutura da árvore, assim como o operador responsável por transformar a árvore geradora em árvore de Steiner.

Função de Aptidão: Toda a população é composta por árvores geradoras e para calcular a aptidão de um indivíduo I_1 com p pontos fixos, seu cromossomo é copiado para um indivíduo temporário I_{temp} , que em seguida tem sua topologia de árvore geradora transformada em uma

topologia de árvore de Steiner cheia por meio de *steinerizações locais* que inserem $p - 2$ pontos Steiner. As coordenadas dos pontos Steiner de I_{temp} são ajustadas para posições que tendem ao ótimo por meio do algoritmo de Smith (1992) e o comprimento da árvore é calculado por meio da distância euclidiana. Ao final, a aptidão do indivíduo I_{temp} é aplicada em I_1 e I_{temp} é excluído.

Inicialização: O algoritmo de Prim (1957) cria uma árvore geradora mínima (MST) de forma gananciosa. A partir de um nó inicial é acrescentado repetidamente a aresta de menor custo que une um novo vértice à árvore em crescimento. Quando a escolha das novas arestas é realizada de forma aleatória e não de acordo com seu custo, o algoritmo é chamado de *PrimRST* (RAIDL; JULSTROM, 2003).

Para criar a população inicial o GA utiliza o algoritmo *PrimRST*. Raidl e Julstrom (2003) mostram que o algoritmo *PrimRST* retorna árvores com o menor diâmetro médio quando comparado com *KruskalRST* e *RandWalkRST*. O diâmetro de uma árvore é o número de arestas existente no caminho mais longo de uma árvore. Quanto menor o diâmetro, mais uma árvore se parece com uma estrela, ou seja, seus vértices tendem a unir diversas arestas.

Seleção: Neste algoritmo é utilizado a *seleção por torneio* em que dois indivíduos diferentes da população são selecionados de forma aleatória e o mais apto é escolhido. Como esse é um problema de minimização, em que se busca uma árvore de menor distância, ganha o torneio o indivíduo que possuir o menor valor de aptidão (comprimento da árvore).

Após os indivíduos escolhidos passarem por cruzamento e mutação, é necessário remover um indivíduo da população para que o indivíduo filho possa ser adicionado. Inicialmente é definido que o pai menos apto deve ser substituído.

Cruzamento: Para fornecer uma boa hereditariedade um operador de cruzamento deve criar uma árvore geradora descendente que seja formada principalmente ou inteiramente por arestas encontradas nos pais da prole (RAIDL; JULSTROM, 2003) e como a árvore gerada não tem restrição de grau esse objetivo é facilmente alcançado para o algoritmo proposto.

Neste trabalho são propostos dois operadores de cruzamento inspirados pelo cruzamento para árvores mostrado por Raidl e Julstrom (2003), que une duas árvores e aplica um algoritmo como o de Prim (1957) ou Kruskal (1956) para selecionar arestas de forma aleatória e assim criar uma nova solução. Ambos os operadores são gulosos, ou seja, tendem a preservar arestas de menor custo ao invés de as escolher aleatoriamente. Eles são aplicados sobre o grafo $G_{cr} = (V, T_1 \cup T_2)$, sendo T_1 e T_2 o conjunto de arestas das árvores progenitoras, isto é, após a união de duas soluções candidatas o operador de cruzamento escolhe quais são as arestas que

irão compor a solução filho. Essa operação pode ser vista por meio da Figura 19.

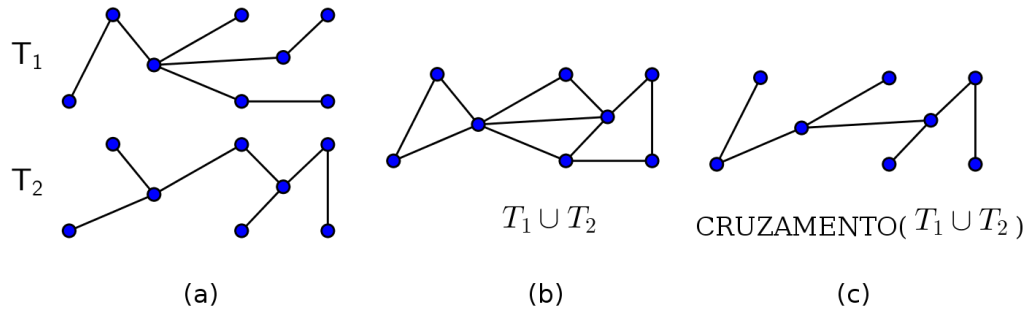


Figura 19: Operador de cruzamento para árvores

Na Figura 19(a) é apresentado duas árvores T_1 e T_2 . A Figura 19(b) exibe uma árvore obtida por meio da união do conjunto de arestas de T_1 e T_2 . Na Figura 19(c) é mostrada a árvore resultante após a aplicação de um dos algoritmos de cruzamento na árvore da Figura 19(b).

O primeiro operador de cruzamento é baseado no algoritmo de Prim (1957) e constrói uma solução adicionando novos vértices a uma árvore em formação de forma relativamente gananciosa. Após inserir um vértice aleatório para iniciar a solução, os demais vértices são inseridos iterativamente e a cada iteração somente a primeira ou a segunda aresta de menor custo pode ser adicionada. O operador é mostrado no Algoritmo 5.

Algoritmo 5: Cruzamento *Prim* relativamente guloso

```

1   $G(V, E) \leftarrow G_1 \cup G_2$ ;                                     // pais
2   $T \leftarrow \emptyset$ ;
3  escolhe um nó inicial qualquer  $s \in V$ ;
4   $C \leftarrow \{s\}$ ;                                           // conjunto de nós conectados
5   $A \leftarrow \{e \in E \mid e = (s, v), v \in V\}$ ;             // arestas elegíveis
6  enquanto  $C \neq V$  faça
7       $rndG \leftarrow$  número aleatório  $\in \{1, 2\}$ ;
8      escolhe a  $rndG^o$  aresta  $(u, v) \in A, u \in C$  de menor custo;
9       $A \leftarrow A - \{(u, v)\}$ ;
10     se  $v \notin C$  então                                       // conecta v na árvore parcial
11          $T \leftarrow T \cup (u, v)$ ;
12          $C \leftarrow C \cup \{v\}$ ;
13          $A \leftarrow A \cup \{e \in E \mid e = (v, w) \wedge w \notin C\}$ ;
14     fim
15 fim
16 retorna  $T$ ;
```

O segundo operador cria o indivíduo filho utilizando uma heurística similar ao algoritmo de Kruskal (1956). Inicialmente o filho recebe todas as arestas que são comuns em ambos os pais e em seguida as demais arestas são adicionadas iterativamente com base em um torneio

entre duas arestas aleatórias, uma de cada pai. O torneio seleciona a aresta de menor custo e se sua adição não gerar ciclos ela é incorporada à árvore. O Algoritmo 6 mostra esse operador.

Algoritmo 6: Cruzamento *Kruskal* relativamente guloso

```

1  $G(V, E) \leftarrow G_1 \cup G_2$ ; // pais
2  $T \leftarrow \emptyset$ ;
3 para todas as arestas  $e(u, v) \in E$  faça
4   se  $(e \in E_1) \wedge (e \in E_2)$  então // arestas em comum
5      $T \leftarrow T \cup \{(u, v)\}$ ;
6      $E \leftarrow E - \{(u, v)\}$ ;
7   fim
8 fim
9 enquanto  $|T| < |E| - 1$  faça // arestas restantes
10  escolhe uma aresta  $(u, v) \in (E \cap E_1)$ ;
11  escolhe uma aresta  $(x, z) \in (E \cap E_2)$ ;
12  se  $distancia(u, v) < distancia(x, z)$  então
13     $(a, b) \leftarrow \{(u, v)\}$ ;
14  senão
15     $(a, b) \leftarrow \{(x, z)\}$ ;
16  fim
17   $E \leftarrow E - \{(a, b)\}$ ;
18  se  $(a, b)$  ainda não está conectado em  $T$  então
19     $T \leftarrow T \cup \{(a, b)\}$ ;
20  fim
21 fim
22 retorna  $T$ ;

```

Ambos os operadores favorecem a exploração, visto que somente as arestas que compõem os pais são utilizadas para criar o filho, não sendo possível a geração de “novas” arestas.

Mutação: Para problemas baseados em árvores, a menor alteração possível em um indivíduo é a substituição de uma aresta por outra aresta viável (RAIDL; JULSTROM, 2003). Dois operadores de mutação gulosos são apresentados, sendo que o primeiro consegue gerar alterações de maior impacto na solução favorecendo a exploração e o segundo cria um descendente que possui uma distância menor para o pai, gerando assim um vizinho próximo.

O primeiro operador escolhe uma aresta aleatoriamente e a remove, o que resulta em duas subárvores desconectadas. Em seguida é selecionado um vértice de cada parte para formar uma nova aresta, e assim reconectar as subárvores. Dentre as arestas candidatas para reconexão, é escolhida aleatoriamente entre a primeira ou a segunda aresta de menor custo. A Figura 20 ilustra o funcionamento desse operador.

Na Figura 20(a) é mostrado um exemplo de árvore a qual será aplicada o primeiro

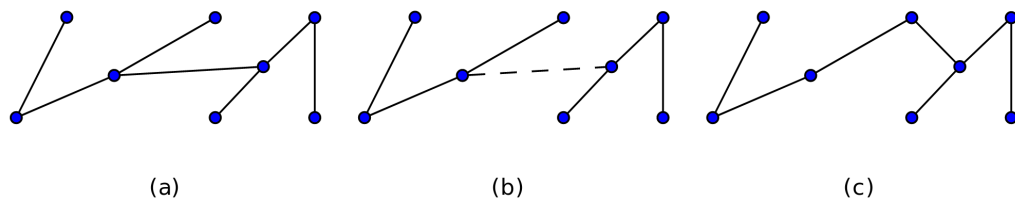


Figura 20: Primeiro operador de mutação (*Rem*)

operador de mutação. Na árvore da Figura 20(b) é removida uma aresta aleatória mostrada por meio da linha tracejada. A Figura 20(c) mostra a árvore resultante após a adição de uma nova aresta que reconecta as subárvores.

O segundo operador escolhe de forma aleatória um vértice $v1$ e um de seus vértices adjacente $v2$, desconecta $v1$ de $v2$ e o reconecta ao vértice adjacente a $v2$ que está mais próximo de $v1$, denominado $v3$. Esse operador cria pequenas modificações na topologia, gerando descendentes próximos do pai. A Figura 21 mostra essa operação.

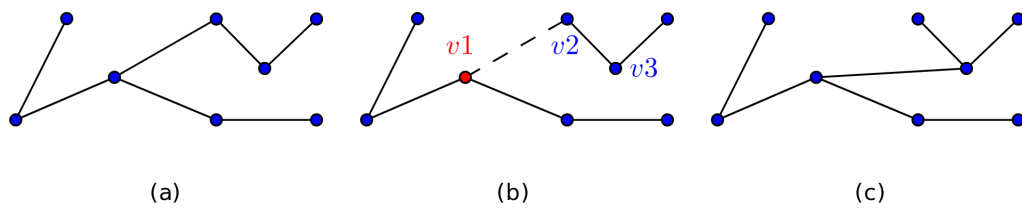


Figura 21: Segundo operador de mutação (*Adj*)

A Figura 21(a) mostra a árvore em que será aplicado o segundo operador de mutação. Na árvore da Figura 21(b) um vértice $v1$ e um de seus vértices incidentes $v2$ é escolhido de forma aleatória, $v1$ é desconectado de $v2$ e reconectado ao vértice $v3$, que é adjacente a $v2$. A Figura 21(c) mostra a árvore resultante após a mutação.

9 EXPERIMENTOS COMPUTACIONAIS

Para avaliar a capacidade do GA de fornecer soluções de boa qualidade, ESTPs da biblioteca de *benchmark OR-Library* (BEASLEY, 1990) são utilizados. Cada problema possui 15 instâncias e para as comparações são utilizadas as soluções ótimas dos problemas com 10, 20, 30, 40, 50, 60, 70, 80, 90 e 100 pontos em $\mathbb{R}^2$.

Os algoritmos propostos foram implementados na linguagem C++, compilados com o GCC 8.2.1, e os experimentos foram executados em um computador com processador Intel Core i5-5200U de 2.20GHz, 4 Gb de memória com o sistema operacional Manjaro Linux 18.0.2 para 64 bit. Para o GA somente um Core é utilizado, já para o DE é habilitado suporte a processamento *multicore* por meio da biblioteca *OpenMP*.

A qualidade das soluções é calculada por meio do percentual de redução da solução sobre a MST, então, sendo T uma solução obtida para um conjunto de pontos, a qualidade de T é dada por $Q = ((l(MST) - l(T)) * 100) / l(MST)$. Como os algoritmos são de natureza estocástica, em um experimento cada instância é processada por trintas vezes para que se possa calcular a média.

9.1 EVOLUÇÃO DIFERENCIAL

9.1.1 ESTRATÉGIAS DE MUTAÇÃO

Existem diversas formas de implementar o operador de mutação para o DE (PRICE et al., 2005). A estratégia de mutação apresentada na Seção 7.2 é chamada de “mutação aleatória” e é utilizada no DE clássico. Para representar diversas variantes de estratégias de mutação, a notação $DE/x/y/z$ é comumente utilizada na literatura de forma que x determina o vetor a ser mutado, podendo ser “*rand*” que significa um vetor da população escolhido aleatoriamente ou “*best*” que representa o vetor mais apto da população, y é a quantidade de vetores de diferença usados na mutação e z indica o operador de cruzamento empregado, sendo “*bin*” caso seja binomial, ou “*exp*” se for exponencial.

Com a finalidade de investigar os efeitos de diferentes estratégias de mutação, os primeiros experimentos testam o desempenho de seis esquemas de mutação com operador de cruzamento binomial (mostrado na Seção 7.2). As estratégias analisadas são mostradas na Tabela 1.

Mutação	Notação
$X_i^{g+1} = X_a^g + F(X_b^g - X_c^g)$	$DE/rand/1$
$X_i^{g+1} = X_{best}^g + F(X_b^g - X_c^g)$	$DE/best/1$
$X_i^{g+1} = X_a^g + F_1(X_b^g - X_c^g) + F_2(X_d^g - X_e^g)$	$DE/rand/2$
$X_i^{g+1} = X_{best}^g + F_1(X_b^g - X_c^g) + F_2(X_d^g - X_e^g)$	$DE/best/2$
$X_i^{g+1} = X_a^g + F_1(X_b^g - X_a^g) + F_2(X_c^g - X_d^g)$	$DE/ctor/1$
$X_i^{g+1} = X_a^g + F_1(X_{best}^g - X_a^g) + F_2(X_b^g - X_c^g)$	$DE/ctob/1$

Tabela 1: Estratégias de mutação para evolução diferencial analisadas

Na Tabela 1, X_i^{g+1} representa o *trialvector*, em que i é o índice do vetor e g denota a geração. Sendo Np a quantidade de indivíduos, os inteiros $a, b, c, d, e \in [1, Np]$ são randômicos e o inteiro $best$ é o índice do indivíduo mais apto da população. Os parâmetros $F, F1, F2 \in [0, 2]$ são os fatores de escala dos operadores. A notação *ctor* se refere à estratégia de mutação “current-to-rand” e a notação *ctob* refere-se à “current-to-best”, em que o vetor atual é movido em direção a outro vetor (*rand* ou *best*) antes de ser perturbado por uma diferença escalonada.

Nesses experimentos os esquemas de mutação são testados com a 15^o instância de 40 pontos e a 1^o instância de 50 pontos da *OR-Library*. Quanto à parametrização inicial, é estabelecido que o tamanho da população é $Np = 25$, os parâmetros $F, F1, F2$ utilizados nas mutações e Cr que representa a taxa de cruzamento são definidos como 0.5, e em ambos os experimentos o DE executa por 30 gerações. As Figuras 22 e 23 mostram a análise de convergência média de 30 testes para a instância de 40 e 50 respectivamente.

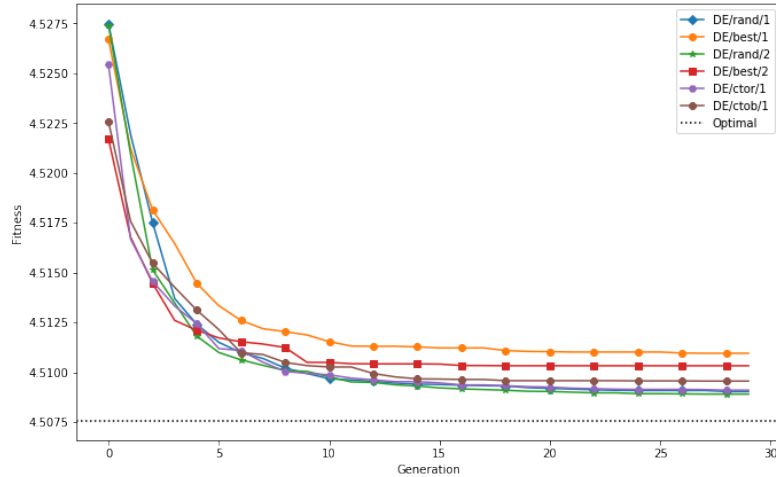


Figura 22: Estratégias de mutação na instância 40.15

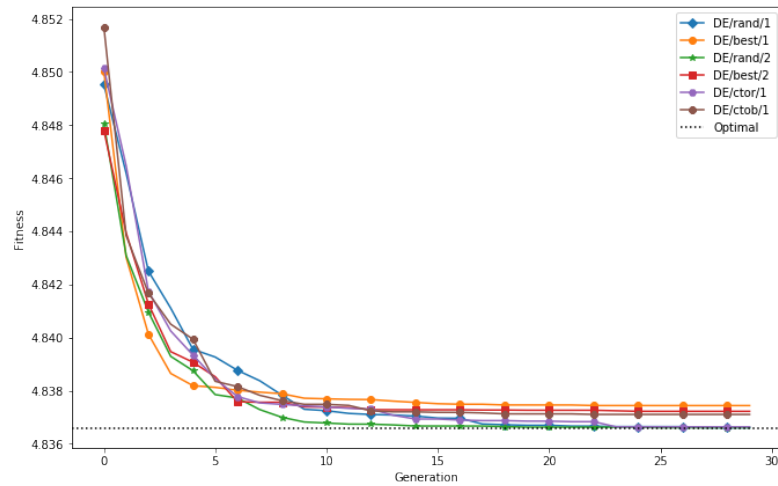


Figura 23: Estratégias de mutação na instância 50.01

A Figura 22 e 23 apresentam indícios de que a estratégia $DE/rand/2$ tende a convergir mais rápido que as demais. Para dar seguimento a análise a Tabela 2 apresenta o percentual de redução médio ($\bar{P}$) e o desvio padrão (Std) dos testes efetuados com cada estratégia de mutação.

Estratégia de mutação	$\bar{P} \pm Std$ da instância	
	40.15	60.01
$DE/rand/1$	3.167 ± 0.016	2.638 ± 0.000
$DE/best/1$	3.124 ± 0.059	2.621 ± 0.038
$DE/rand/2$	3.168 ± 0.016	2.638 ± 0.000
$DE/best/2$	3.139 ± 0.042	2.625 ± 0.048
$DE/ctor/1$	3.166 ± 0.016	2.637 ± 0.003
$DE/ctob/1$	3.154 ± 0.029	2.627 ± 0.030

Tabela 2: Percentuais de redução obtidos com as estratégias de mutação

Na Tabela 2, para a instância de 40 pontos a melhor estratégia é $DE/rand/2$ que apresenta média de minimização de 3.168% sobre a MST, seguido de $DE/rand/1$ com 3.167% e $DE/ctor/1$ com 3.166%. Apesar de serem boas soluções, nenhuma das estratégias atingiu a média ótima de 3.194% de redução. Na instância de 50 pontos, $DE/rand/1$ e $DE/rand/2$ apresentam a mesma taxa de redução de 2.638%, seguidos de $DE/ctor/1$ com 2.637%. Levando vantagem em qualidade de solução em um experimento, e convergindo mais rápido em dois, $DE/rand/2$ mostra-se a melhor estratégia para dar seguimento aos experimentos.

9.2 ALGORITMO GENÉTICO

9.2.1 OPERADORES DE CRUZAMENTO E MUTAÇÃO

Os primeiros testes realizados investigam o desempenho dos operadores genéticos de cruzamento e mutação em um GA de estado estacionário. A utilização desse modelo de GA, ao invés do GA populacional tende a favorecer o tempo computacional e o consumo de memória, visto que o GA populacional cria uma população nova a cada geração, ao que, o de estado estacionário cria somente um novo indivíduo. Para os experimentos, os operadores de cruzamento foram chamados de “*Prim*” e “*Kruskal*”, pois são baseados nesses algoritmos. O operador de mutação que remove uma aresta e divide a árvore em subárvores e depois a reagrupa é chamado de “*Rem*” e o operador de mutação que cria arestas com base em vértices adjacentes é chamado de “*Adj*”.

O primeiro experimento mostra o desempenho dos operadores genéticos conjugados. Para a abordagem inicial, foi definida uma população de 50 indivíduos, e como se trata de um GA de estado estacionário, a cada geração dois indivíduos são selecionados para cruzamento dando origem a um filho que sofre mutação e em seguida substitui o pai menos apto. O primeiro experimento é aplicado sob a 1^o instância do problema de 100 pontos por 600 gerações. O segundo experimento é aplicado sob todas as instâncias até que se atinja um resultado ótimo ou não haja melhora na solução por 100 gerações (estagnação).

Para ambos os experimentos, cada instância é testada por 30 vezes para que se obtenha a média. A Figura 24 apresenta a curva de convergência do primeiro experimento e a Figura 25 mostra o percentual de redução do segundo experimento.

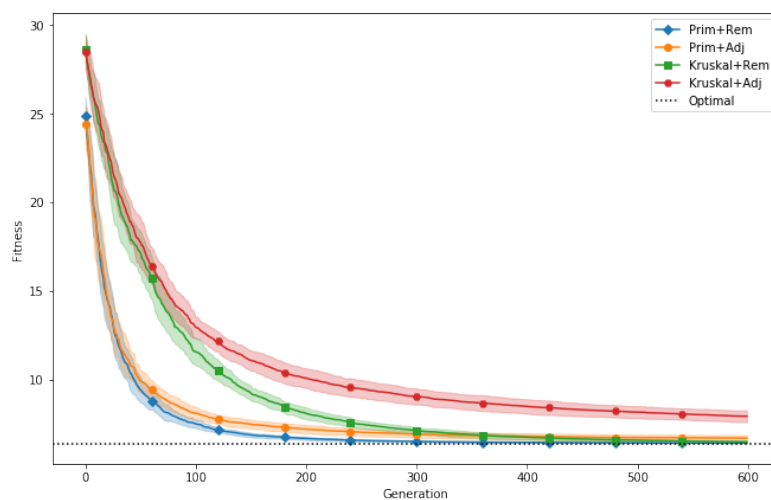


Figura 24: Operadores no GA de estado estacionário. Instância: 100.1

A Figura 24 mostra que o operador de cruzamento *Prim* converge mais rápido do que *Kruskal*, o que se deve ao fato de *Prim* ser mais ganancioso, pois para selecionar uma nova aresta para compor uma solução, *Prim* compara o custo de diversas arestas e *Kruskal* verifica somente duas. Como *Kruskal* preserva as arestas em comum dos dois pais, a princípio muitas arestas ruins podem ser mantidas nas soluções. Quanto aos operadores de mutação, *Rem* converge mais rápido que *Adj*, pois gera soluções mais distantes dos pais favorecendo a exploração e consequentemente a manutenção da diversidade populacional.

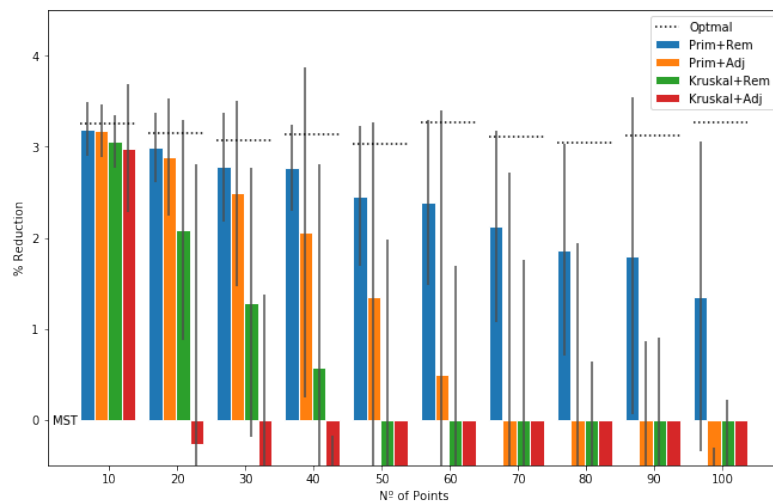


Figura 25: Operadores no GA de estado estacionário

A Figura 25 mostra o percentual de redução médio com relação à árvore geradora mínima (MST). No teste inicial o operador de cruzamento *Prim* se mostra melhor que *Kruskal* e o operador de mutação *Rem* melhor que *Adj*. A única combinação de operadores que consegue obter uma média positiva para todos os conjuntos de pontos é a combinação *Prim+Rem*. A combinação *Prim+Adj* só possui resultados positivos até 60 pontos, *Kruskal+Rem* até 40 pontos e *Kruskal+Adj* só consegue bons ganhos com instâncias de 10 pontos.

Os experimentos iniciais apresentam resultados com qualidade muito inferior aos relatados pela literatura, entretanto são necessários para entender o comportamento do GA e dos operadores propostos. O operador genético mais ganancioso e o operador de mutação que cria indivíduos mais distantes se mostraram eficientes, isso se deve ao fato de que uma alta ganância gera árvores próximas à geradora mínima e a criação de indivíduos distantes favorece a manutenção da diversidade populacional.

9.2.2 SOBREPOSIÇÃO DE GERAÇÕES

Na primeira abordagem é utilizado um GA de “estado estacionário” em que somente um indivíduo é substituído em uma população de tamanho fixo N a cada geração. Ao aumentar a quantidade de substituições por geração sem que se crie uma população temporária como ocorre no GA populacional, novos indivíduos se misturam a indivíduos de gerações mais antigas, ou seja, há uma sobreposição de gerações, o que leva o algoritmo a ser chamado de “*generation gap algorithm*” (JONG; SARMA, 1992). O primeiro estudo que tenta avaliar os efeitos da sobreposição de gerações é apresentado por Jong (1975). Nesse estudo um parâmetro G denominado “intervalo de geração” (“*generational gap*” em inglês) é utilizado para controlar a fração da população a ser substituída a cada geração.

Sendo g o percentual de indivíduos substituídos em uma população de tamanho n em uma geração, o terceiro experimento mostra o efeito da sobreposição de gerações nos operadores genéticos por meio da variação de g com $n = 50$. Nesse experimento foram testados os dois operadores de cruzamento (*Prim* e *Kruskal*) conjugados com o melhor operador de mutação do teste anterior (*Rem*). O parâmetro g foi testado com 10, 25, 50, 75 e 90% e o pior indivíduo da população é substituído pelo filho. A Figura 26 mostra o resultado desse experimento aplicado sob a 1ª instância do problema de 100 pontos por 30 gerações em 30 testes.

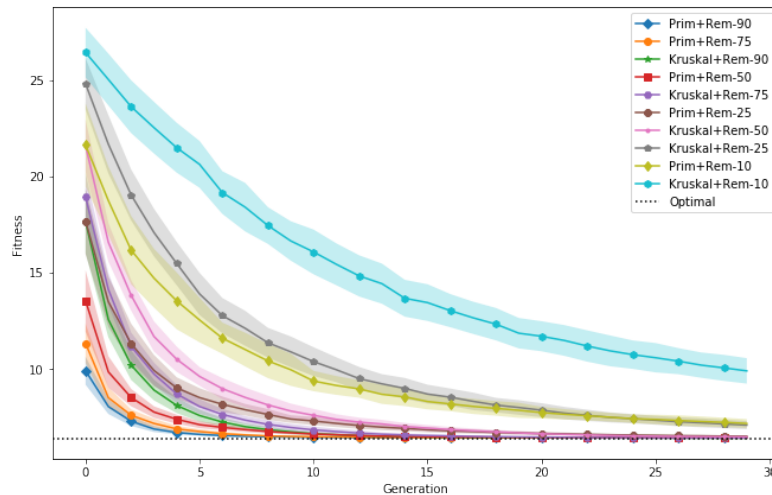


Figura 26: Operadores no *generation gap algorithm*. Instância: 100.1

Na Figura 26, testes com $g > 50$ mostram que uma rápida convergência é alcançada com um alto índice de sobreposição de gerações. Os testes *Prim+Rem90* e *Kruskal+Rem90* apresentam os melhores resultados para um número pequeno de gerações, porém também são detentores de um maior tempo médio de execução. O teste *Prim+Rem90* obtém 3.16% de redução, *Kruskal+Rem90* obtém 3.15% e os demais testes apresentam reduções inferiores. O

tempo de execução é proporcional a g , ou seja, quanto maior o valor de g , maior o tempo médio de execução.

No quarto experimento é utilizado 90% de substituição por geração em uma população de 50 indivíduos. A quantidade máxima de gerações é definida como 5 vezes a quantidade de pontos do problema, estabelecendo uma proporcionalidade com o tamanho do problema. Os algoritmos são aplicados sob todas as instâncias até que se atinja um resultado ótimo ou fique estagnado por 50 gerações. Os testes iniciais foram realizados com os operadores de cruzamento *Prim* e *Kruskal* conjugados com o operador de mutação *Rem*. Com a finalidade de diversificar os métodos de exploração do espaço de busca, também foi testado uma abordagem em que os dois operadores de mutação (*Adj+Rem*) são aplicados no indivíduo originado do cruzamento. Os resultados médios de 30 testes para as variantes *Prim+Rem*, *Kruskal+Rem*, *Prim+Adj+Rem* e *Kruskal+Adj+Rem* são vistos na Figura 27.

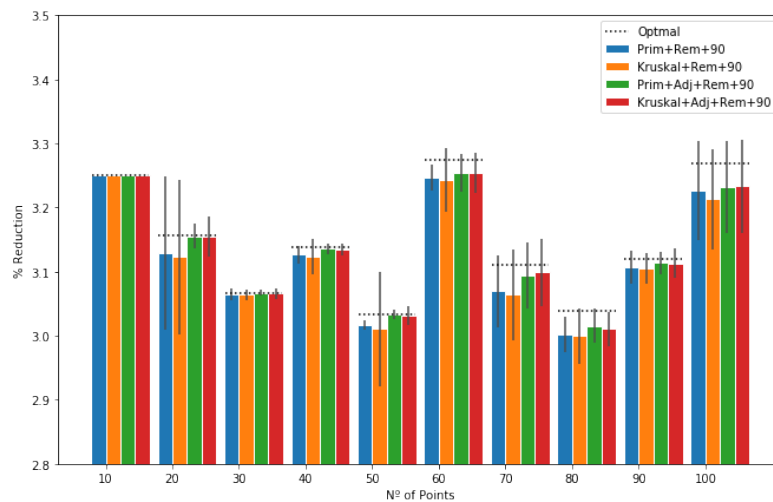


Figura 27: Operadores no *generation gap algorithm*

Na Figura 27 é possível ver que as abordagens que utilizam os dois operadores de mutação atingem melhores resultados. O Operador de cruzamento *Prim* alcança melhores resultados na maioria dos problemas quando comparado a *Kruskal*. Como o pior indivíduo da população é eliminado a cada aplicação dos operadores genéticos, a diversidade populacional também diminui rapidamente e pode levar o algoritmo a estagnação.

9.2.3 REDEFINIÇÃO POPULACIONAL

Para aumentar a diversidade populacional o quinto teste visa investigar o efeito da redefinição da população em caso de estagnação. Para isso a melhor abordagem do experimento anterior (*Prim+Adj+Rem+90*) recebe dois modelos de redefinição populacional. A execução

finaliza se a aptidão não melhorar por 25 gerações seguidas e a redefinição populacional ocorre se desvio padrão da população for inferior à 0.03. O primeiro modelo de redefinição denominado “*RstElitist*” mantém o melhor indivíduo e substitui os demais por novas soluções. O segundo modelo é denominado “*RstBest*” e substitui o melhor indivíduo da população por um novo. A Figura 28 apresenta os resultados.

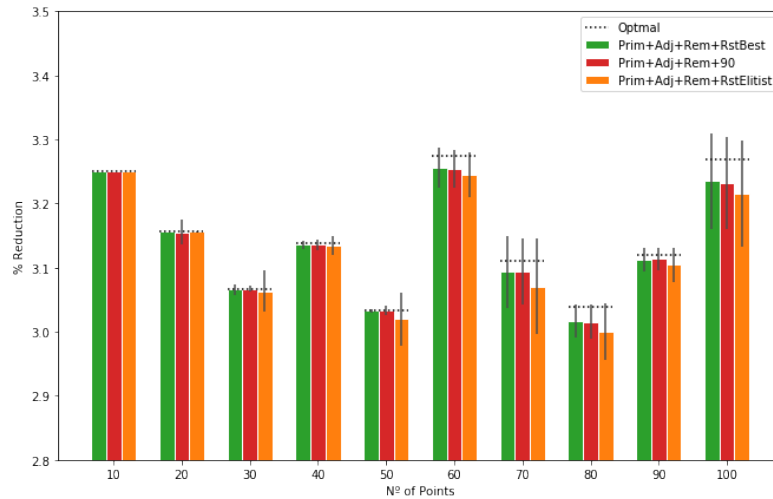


Figura 28: Operadores no *generation gap algorithm* com redefinição

Na Figura 28 a abordagem *Prim+Adj+Rem+RstBest* obtém um resultado melhor que *Prim+Adj+Rem+RstElitist*, mostrando uma melhora de 0.01% no percentual de redução médio e de 30% no tempo de execução médio, quando comparada à abordagem sem redefinição populacional. Como a seleção é por torneio de dois indivíduos, a retirada do melhor indivíduo permite que outros indivíduos com potencial possam se desenvolver. A remoção de muitos indivíduos não é benéfica, pois muito material genético é perdido, o que faz com que a abordagem *Prim+Adj+Rem+RstElitist* tenha resultado inferior à abordagem que não utiliza um método de redefinição.

9.3 ANÁLISE DE DESEMPENHO

Para estabelecer uma comparação entre os algoritmos desenvolvidos, os mesmos são testados com a configuração com a qual obtiveram melhor desempenho nos testes apresentados nesse capítulo. Tanto o DE quando o GA utilizam população com $Np = 50$ indivíduos. O DE utiliza a estratégia de mutação $DE/rand/2/bin$ com $F1, F2, Cr = 0.5$. O GA utiliza a configuração *Prim+Adj+Rem+RstBest* com 90% de substituição populacional por geração e caso a população atingir um desvio padrão inferior à 0.03, a técnica de redefinição populacional *RstBest* é aplicada. Nos dois algoritmos a execução é encerrada ao atingir o ótimo, executar por

g gerações, ou se estagnar por e gerações consecutivas. Sendo n a quantidade de pontos, para o DE $g = 50$ e $e = 10$, e para o GA $g = n * 10$ e $e = 50$.

Para o teste final são utilizadas os problemas com 10 à 100 pontos da *OR-Library* (BEASLEY, 1990). Cada problema possui 15 instâncias, e para cada instância são executados 30 testes e calculado o percentual de redução de cada um. Sendo P_x um problema com x pontos; $I_{x,y}$ a instância y de um problema com x pontos, e $T_{x,y}^z$ o percentual de redução de um teste z da instância y de um problema de x pontos; a média por instância $\overline{I_{x,y}} = (\sum_{z=1}^{30} T_{x,y}^z)/30$, logo a média por pontos é $\overline{P} = (\sum_{y=1}^{15} \overline{I_{x,y}})/15$. Sendo $Ib_{x,y} = \max(T_{x,y}^z | 1 \leq z \leq 30)$ o melhor teste de uma instância y com x pontos, o melhor valor de aptidão por ponto é $P_{best} = (\sum_{y=1}^{15} Ib_{x,y})/15$. O desvio padrão por ponto é obtido por meio de $Std = \sqrt{SSe/(29 * 15)}$, sendo $SSe = \sum_{y=1}^{15} \sum_{z=1}^{30} (T_{x,y}^z - \overline{I_{x,y}})^2$ (MONTGOMERY, 2006).

Os resultados são comparados ao algoritmo de Beasley e Goffnet (1994) e Laarhoven e Ohlmann (2010), pois ambos são de natureza estocástica e apresentam resultados competitivos. O estudo de Beasley e Goffnet (1994) é utilizado para contextualização histórica e o de Laarhoven e Ohlmann (2010) é o trabalho mais recente que viabiliza uma comparação de resultados por meio de uma biblioteca de *benchmark*. A Tabela 3 e a Figura 29 apresentam os resultados.

n	Beasley e Goffinet (1994)		Laarhoven e Ohlmann (2010)		Evolução Diferencial Híbrido		Algoritmo Genético Híbrido		Opt.
	$\bar{P} \pm Std$	P_{best}	$\bar{P} \pm Std$	P_{best}	$\bar{P} \pm Std$	P_{best}	$\bar{P} \pm Std$	P_{best}	
10	3.22±1.875	3.25	3.22±0.151	3.25	3.25±0.000	3.25	3.25±0.000	3.25	3.25
20	3.12±0.972	3.15	3.13±0.058	3.16	3.16±0.000	3.16	3.16±0.000	3.16	3.16
30	2.95±0.754	3.07	3.03±0.068	3.07	3.06±0.057	3.07	3.07±0.004	3.07	3.07
40	2.97±0.633	3.14	3.11±0.051	3.14	3.14±0.018	3.14	3.14±0.000	3.14	3.14
50	2.92±0.423	3.03	3.00±0.049	3.03	3.03±0.010	3.03	3.03±0.014	3.03	3.03
60	3.18±0.371	3.27	3.23±0.058	3.27	3.26±0.024	3.27	3.26±0.031	3.27	3.27
70	2.95±0.374	3.11	3.08±0.050	3.11	3.10±0.031	3.11	3.09±0.055	3.11	3.11
80	2.92±0.686	3.03	2.98±0.048	3.03	3.00±0.032	3.03	3.02±0.025	3.03	3.04
90	2.95±0.502	3.11	3.04±0.062	3.11	3.08±0.033	3.12	3.12±0.014	3.12	3.12
100	3.07±0.337	3.26	3.19±0.061	3.25	3.19±0.045	3.27	3.23±0.065	3.27	3.27

Tabela 3: Análise de desempenho

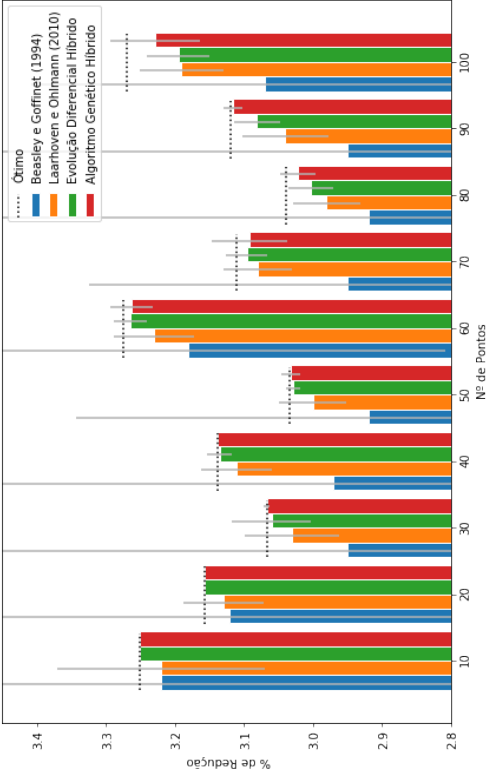


Figura 29: Análise de desempenho

Na Tabela 3, “ n ” representa o número de pontos fixo do problema, “ P_{best} ” o melhor percentual de redução, “ $\bar{P} \pm \text{Std.}$ ” o percentual de redução médio, e “Opt.” a média de redução ótima disponibilizada na *Or-Library*. A Figura 29 apresenta uma comparação gráfica dos resultados agrupados pela quantidade de pontos.

Analisando os resultados é possível observar que o Algoritmo Genético apresenta resultados melhores que as demais abordagens. No que diz respeito ao percentual médio de redução ($\bar{P}$), o DE apresenta resultados ótimos para 10 e 20 pontos, muito próximo do ótimo para 40 e 50 pontos e 0.01% inferior ao ótimo para 30, 60 e 70 pontos. O GA apresenta resultados ótimos ou muito próximo para os testes realizados com 10, 20, 30, 40, 50 e 90 pontos e redução de 0.01% à 0.04% inferior ao ótimo para o restante. Quanto ao melhor percentual de redução (P_{best}), o DE possui a mesma média geral que a abordagem de Laarhoven e Ohlmann (2010), atingindo o ótimo em 70% dos problemas; já no GA a redução só não atinge o ótimo para 80 pontos, em que o algoritmo não encontra o melhor solução em nenhuma das 30 execuções da 2°, 3° e 4° instância, entretanto, o resultado é 0.01% inferior ao ótimo.

Para comparar a média do DE e GA com o algoritmo de Laarhoven e Ohlmann (2010) foi utilizado um *teste-T* pareado, visto que os valores estão em uma distribuição normal. Ambos os testes resultaram em $p_{value} \approx 0$, mostrando que há uma diferença estatística significativa entre os algoritmos desenvolvidos e a abordagem de Laarhoven e Ohlmann (2010). Ao comparar o DE com o GA o resultado é $p_{value} \approx 0.1$, logo, como $p_{value} > 0.05$, considera-se que não há uma diferença estatística significativa entre as médias dos algoritmos desenvolvidos.

10 CONSIDERAÇÕES FINAIS

O ESTP é um problema difícil de ser resolvido, e como métodos exatos podem ser computacionalmente exigentes, esse trabalho apresenta dois algoritmos evolutivos híbridos para o ESTP em $\mathbb{R}^2$. A primeira heurística utiliza um algoritmo evolução diferencial (DE) e conta com um método que permite construir árvores de Steiner a partir de um conjunto de pontos (fixos e pontos Steiner) dispostos no plano. Já a segunda abordagem se baseia em um algoritmo genético (GA) e conta com operadores genéticos gulosos próprios para a estrutura de dados árvore unido a um método que permite a transformação de árvores geradoras em árvores de Steiner.

Para o DE são testados seis estratégias de mutação com duas instâncias da *OR-Library*, e dentre os esquemas comparados, o que obteve maior eficiência foi o operador *DE/rand/2/bin* que utiliza dois vetores de diferença para mutar um vetor aleatório da população.

No caso do GA, inicialmente foi testado um algoritmo de estado estacionário com dois operadores de cruzamento (*Prim* e *Kruskal*) e dois de mutação (*Adj* e *Rem*) com os problemas da *Or-Library*. Os resultados iniciais mostram a ineficiência do operador de mutação *Adj*. A cada geração somente um novo indivíduo é gerado e isso resultou em um desempenho ruim para a maioria das combinações de operadores.

Os experimentos seguintes investigam a sobreposição de gerações por meio de um *generational gap algorithm* em que um novo indivíduo substitui o pior indivíduo da população. Sobreposições maiores se mostraram mais eficientes apesar de consumir mais tempo computacional. A utilização de dois operadores de mutação também se mostrou eficiente, pois permite diferentes formas de explorar o espaço de busca. Testes com 90% de substituição por geração mostram bons resultados com o operador *Prim* aliado a *Ajd* e *Rem*.

Com o *generational gap* a heurística apresentou uma melhora, entretanto, indivíduos dominantes podem tomar conta da população rapidamente, o que levou a investigação de redefinição populacional. Nenhum modelo de redefinição resultou em um aumento de qualidade expressivo, porém, a técnica de retirar o melhor indivíduo em caso de estagnação apresentou uma

melhora de 30% no tempo de execução.

O último experimento utiliza as instâncias de 10 à 100 pontos da *OR-Library* e possibilita comparar a melhor configuração encontrada para os algoritmos desenvolvidos com estudos relevantes da literatura. Ambos os algoritmos empregam diversas técnicas que viabilizam um bom desempenho, como inicialização da população com indivíduos de boa qualidade e correções das soluções durante o processo de evolução para não gerar indivíduos inválidos. Apesar de o DE apresentar bons resultados, os mecanismos implementados no GA se mostram mais eficientes, visto que gera resultados próximos à de soluções ótimas e consome menos tempo computacional.

O fato de o GA ser mais eficiente que o DE não se dá somente pelas diferenças na essência das meta-heurísticas, mas principalmente por conta do modo como o problema é abordado, visando a redução de dimensionalidade do espaço de busca. O emprego de operadores genéticos gulosos fornece pressão seletiva ao algoritmo, favorecendo a construção de soluções próximas a árvores geradoras de custo mínimo. Em determinadas instâncias uma pressão seletiva alta pode levar a busca para mínimos locais, logo, para tentar evitar essa tendência as gerações são sobrepostas para gerar diversidade genética. Em caso de estagnação uma redefinição populacional é empregada para reposicionar a busca em um local próximo. Todas essas técnicas juntas contribuem para que o GA desenvolva soluções de boa qualidade em tempo computacional viável.

Dado a importância do ESTP no mundo real, trabalhos futuros incluem testes com instâncias em $\mathbb{R}^d$ com $d > 2$, a utilização de problemas com um alto número de pontos terminais, a construção de operadores adaptativos e a utilização de populações paralelas.

REFERÊNCIAS

- AREIBI, S.; XIE, M.; VANNELLI, A. An efficient rectilinear Steiner tree algorithm for VLSI global routing. In: **Canadian Conference on Electrical and Computer Engineering 2001. Conference Proceedings**. [S.l.: s.n.], 2001.
- BARREIROS, J. An hierarchic genetic algorithm for computing (near) optimal Euclidean Steiner trees. In: **Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2003)**. [S.l.]: Springer-Verlag, 2003.
- BEASLEY, J. E. OR-library: Distributing test problems by electronic mail. **Journal of the Operational Research Society**, 1990.
- BEASLEY, J. E. A heuristic for Euclidean and rectilinear Steiner problems. **European Journal of Operational Research**, 1992.
- BEASLEY, J. E.; GOFFNET, F. A Delaunay triangulation-based heuristic for the Euclidean Steiner problem. **Networks**, 1994.
- BERETA, M. Baldwin effect and lamarckian evolution in a memetic algorithm for Euclidean Steiner tree problem. **Memetic Computing**, 2018.
- BINIAZ, A.; DASTGHAIBYFARD, G. A faster circle-sweep Delaunay triangulation algorithm. **Advances in Engineering Software**, 2012.
- BOPP, K. **Über das kürzeste Verbindungssystem zwischen vier Punkten**. Tese (Doutorado) — Universität Göttingen, 1879.
- BORŮVKA, O. O jistém problému minimalním. **Práce Moravské Přírodovědecké Společnosti**, 1926.
- BRAZIL, M. et al. **On the Story of Euclidian Steiner Tree Problem**. [S.l.]: Archive for History of Exact Sciences, 2013.
- CAVALIERI, B. F. **Exercitationes Geometricæ**. [S.l.]: Typis Iacobi Montij, 1647.
- CHANG, S. The generation of minimal trees with a Steiner topology. **Journal of the ACM (JACM)**, 1972.
- COCKAYNE, E. J.; SCHILLER, D. G. Computation of Steiner minimal trees. In: **Combinatorics**. [S.l.]: Institute for Mathematics and Applications, 1972.
- COURANT, R.; ROBBINS, H. **What is mathematics?** [S.l.]: London: Oxford University Press, 1941.
- DELBEM, A. C. B. et al. Node-Depth encoding for evolutionary algorithms applied to network design. In: **Genetic and Evolutionary Computation – GECCO 2004**. [S.l.: s.n.], 2004.

- DELBEM, A. C. B.; de LIMA, T. W.; TELLES, G. P. Efficient forest data structure for evolutionary algorithms applied to network design. **IEEE Transactions on Evolutionary Computation**, 2012.
- DEZA, M. M.; DEZA, E. **Encyclopedia of distances**. [S.l.]: Springer Berlin Heidelberg, 2009.
- DIESTEL, R. **Graph Theory**. [S.l.]: Springer, 1997.
- DREYER, D.; OVERTON, M. Two heuristics for the Euclidean Steiner tree problem. **Journal of Global Optimization**, 1998.
- DU, D.; HU, X. **Steiner Tree Problems In Computer Communication Networks**. [S.l.]: World Scientific Publishing Co., Inc., 2008.
- ERCIYES, K. **Guide to graph algorithms sequential, parallel and distributed**. [S.l.]: Springer, 2018.
- EVEN, S. **Graph algorithms**. [S.l.]: Cambridge University Press, 2011.
- FAMPA, M.; ANSTREICHER, K. M. An improved algorithm for computing Steiner minimal trees in Euclidean d-space. **Discrete Optimization**, 2008.
- FERMAT, P. de. Methodus ad disquirendam maximam et minimam et de tangentibus linearum curvarum. In: **Œuvres de Fermat**. [S.l.]: Gauthier-Villars, 1891.
- FORTE, V. L. d. et al. Iterated local search algorithms for the Euclidean Steiner tree problem in n dimensions. **International Transactions in Operational Research**, 2015.
- FORTUNE, S. Voronoi diagrams and Delaunay triangulations. In: **Handbook of Discrete and Computational Geometry**. [S.l.]: CRC Press, Inc., 1997.
- GAREY, M. R.; GRAHAM, R. L.; JOHNSON, D. S. The complexity of computing Steiner minimal trees. **SIAM Journal on Applied Mathematics**, 1977.
- GERGONNE, J. D. Solutions purement géométriques des problèmes de minimis proposés aux pages 196, 232 et 292 de ce volume, et de divers autres problèmes analogues. In: **Annales de Mathématiques pures et appliquées**. [S.l.]: Imprimeur-Libraire pour les Mathématiques, 1810.
- GILBERT, E. N.; POLLAK, H. O. Steiner minimal trees. **SIAM Journal on Applied Mathematics**, 1968.
- GOLDBERG, D. E. **Genetic algorithms in search, optimization and machine learning**. [S.l.]: Addison Wesley, 1989.
- HANAN, M. On Steiner's problem with rectilinear distance. **SIAM Journal on Applied Mathematics**, 1966.
- HEINEN, F. **Über Systeme von Kräften**. [S.l.]: G. D. Bädeker, 1834.
- HOFFMANN, E. Über das kürzeste verbindungssystem zwischen vier punkten der ebene. In: **Program des Königlichen Gymnasiums zu Wetzlar für das Schuljahr von Ostern 1889 bis Ostern 1890**. [S.l.]: Schnitzler, 1890.

- HSIE, M.; WU, M.-Y.; HUANG, C. Y. Optimal urban sewer layout design using Steiner tree problems. **Engineering Optimization**, 2019.
- HWANG, F. K. A linear time algorithm for full Steiner trees. **Operations Research Letters**, 1986.
- HWANG, F. K.; RICHARDS, D. S.; WINTER, P. **The Steiner tree problem**. [S.l.]: Elsevier Science Publishers B. V., 1992.
- HWANG, F. K.; WENG, J. F. The shortest network under a given topology. **J. Algorithms**, 1992.
- JARNÍK, V.; KÖSSLER, O. O minimálních grafeth obeahujících n daných bodů. **Cas. Pěstování Mat. a Fys.**, 1934.
- JESUS, M.; JESUS, S.; MÁRQUEZ, A. Steiner trees optimization using genetic algorithms. In: **Technical report**. [S.l.]: Centro de Simulação e Cálculo, 2004.
- JONG, K. A. D. **An analysis of the behavior of a class of genetic adaptive systems**. Tese (Doutorado) — University of Michigan, 1975.
- JONG, K. A. D.; SARMA, J. Generation gaps revisited. In: **FOGA**. [S.l.: s.n.], 1992.
- JUHL, D. et al. The GeoSteiner software package for computing Steiner trees in the plane: an updated computational study. **Mathematical Programming Computation**, 2018.
- JULSTROM, B. The Blob Code is competitive with edge-sets in genetic algorithms for the minimum routing cost spanning tree problem. **GECCO 2005 - Genetic and Evolutionary Computation Conference**, 2005.
- JUNGNICKEL, D. **Graphs, Networks and Algorithms**. [S.l.]: Springer-Verlag Berlin Heidelberg, 2013.
- KOCH, T.; MARTIN, A. Solving Steiner tree problems in graphs to optimality. **Networks**, 1998.
- KRUPKE, D. et al. Distributed cohesive control for robot swarms: Maintaining good connectivity in the presence of exterior forces. In: **2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)**. [S.l.: s.n.], 2015.
- KRUSKAL, J. B. On the shortest spanning tree of a graph and the traveling salesman problem. **Proc. Amer. Math. Soc.**, 1956.
- LAARHOVEN, J. W.; OHLMANN, J. W. A randomized Delaunay triangulation heuristic for the Euclidean Steiner tree problem in $\mathbb{R}^d$. **Journal of Heuristics**, 2010.
- LUKE, S. **Essentials of Metaheuristics**. [S.l.]: Lulu, 2013.
- LUNDY, M. Applications of the annealing algorithm to combinatorial problems in statistics. **Biometrika**, 1985.
- MELZAK, Z. A. On the problem of Steiner. **Canad. Math. Bull.**, 1961.

- MIGUEZ, E. et al. An improved branch-exchange algorithm for large-scale distribution network planning. **IEEE Transactions on Power Systems**, 2002.
- MITCHELL, M. **An Introduction to Genetic Algorithms**. [S.l.]: MIT Press, 1998.
- MONDAINI, R. P.; OLIVEIRA, N. V. Intramolecular structure of proteins as driven by Steiner optimization problems. In: **18th International Conference on Systems Engineering (ICSEng'05)**. [S.l.: s.n.], 2005.
- MONTGOMERY, D. C. **Design and Analysis of Experiments**. [S.l.]: John Wiley & Sons, Inc., 2006.
- OLIVEIRA, M. A. A. de et al. Heuristic applied to the Euclidean Steiner tree problem with node-depth-degree encoding. In: **Proceedings on the International Conference on Artificial Intelligence (ICAI)**. [S.l.: s.n.], 2014.
- PALMER, C.; KERSHENBAUM, A. An approach to a problem in network design using genetic algorithms. **Networks**, 1995.
- PRICE, K. V.; STORN, R. N.; LAMPINEN, J. A. **Differential Evolution: A Practical Approach to Global Optimization**. [S.l.]: Springer, 2005.
- PRIM, R. C. Shortest connection networks and some generalizations. **Bell System Tech. J.**, 1957.
- RAIDL, G. R.; JULSTROM, B. A. Edge sets: an effective evolutionary coding of spanning trees. **IEEE Transactions on Evolutionary Computation**, 2003.
- ROTHLAUF, F.; GOLDBERG, D. E. **Representations for genetic and evolutionary algorithms**. [S.l.]: Physica-Verlag, 2006.
- SHAMOS, M. I.; HOEY, D. Closest-point problems. In: **Proceedings of the 16th Annual Symposium on Foundations of Computer Science**. [S.l.: s.n.], 1975.
- SIMPSON, T. **The doctrine and application of fluxions**. [S.l.]: John Nourse Publisher, 1750.
- SINCLAIR, D. S-hull: a fast radial sweep-hull routine for Delaunay triangulation. **CoRR**, 2010.
- SMITH, J. M.; LEE, D. T.; LIEBMAN, J. S. An $o(n \log n)$ heuristic for Steiner minimal tree problems on the Euclidean metric. **Networks**, 1981.
- SMITH, W. D. How to find Steiner minimal trees in Euclidean d-space. **Algorithmica**, 1992.
- STORN, R.; PRICE, K. Differential evolution - a simple and efficient heuristic for global optimization over continuous spaces. **Journal of Global Optimization**, 1997.
- THOMPSON, E. A. The method of minimum evolution. **Annals of human genetics**, 1973.
- TORRICELLI, E. De maximis et minimis. In: **Opere di Evangelista Torricelli**. [S.l.]: G. Montanari, 1919.
- WANG, L.; DU, D.-Z. Approximations for a bottleneck Steiner tree problem. **Algorithmica**, 2002.

WARME, D. M. **Spanning trees in hypergraphs with applications to Steiner trees**. Tese (Doutorado) — University of Virginia, 1998.

WHITLEY, D. GENITOR : a different genetic algorithm. **Proceedings of the Rocky Mountain conference on artificial intelligence, 1988**, 1988.

WINTER, P. An algorithm for the Steiner problem in the Euclidean plane. **Networks**, 1985.

WINTER, P.; ZACHARIASEN, M. Euclidean Steiner minimum trees: An improved exact algorithm. **Networks**, 1997.

YANG, B. A hybrid evolutionary algorithm for the Euclidean Steiner tree problem using local searches. In: **Knowledge-Based Intelligent Information and Engineering Systems**. [S.l.: s.n.], 2006.

ZHOU, G.; GEN, M. A note on genetic algorithms for degree-constrained spanning tree problems. **Networks**, 1997.